Chapter 5

ReactJS with AI Features

Table of Contents

Chapter 4 1

React JS with AI Features 1

4.1. Introduction 2

4.2. Setting up a React Environment 4

4.3. Understanding React Components 9

4.4. Function components in React 10

4.4.1. JSX in React 10

4.4.2. How to style with JSX 12

4.4.3. JS variables in JSX 15

4.4.4. Using Variables in React JSX 16

4.4.5. React App usings semantic UI styling 17

4.5. React Routing: Navigation in Single Page Applications (SPA) 22

4.5.1. Setting up the router 23

4.5.2. 25

4.6. Properties, props, system in React JS 33

4.6.1. Adding props children in React 41

4.6.2. Adding event listener 48

4.7. Application of React States 62

4.7.1. React States Components 62

4.7.2. Event handler in React 72

4.7.3. Use of state 73

4.8. Shopping cart using react State 79

Part 1: Setting up the React environment and Starter Structure 80
Part 2. Create the product list 82
4.9. React Forms using States 93
4.9.1. Handling Forms 93
4.9.2. The useState statement 93
4.9.3. Initialize useState 94
4.9.4. Submitting Forms 96
4.9.5. Multiple Input Fields 98

5.1. Introduction

React is a JavaScript library for building user interfaces, UI. React, sometimes referred to as a frontend JavaScript framework, is a JavaScript library created by Facebook. It is a tool for building UI components.

React is used to build single-page applications. It also allows us to create reusable UI components.

How does React Work?

- React creates a VIRTUAL DOM in memory.
- Instead of manipulating the browser's DOM directly, React creates a virtual DOM in memory, where it does all the necessary manipulating, before making the changes in the browser DOM.
- React only changes what needs to be changed!
- React finds out what changes have been made, and changes **only** what needs to be changed.

React.JS History

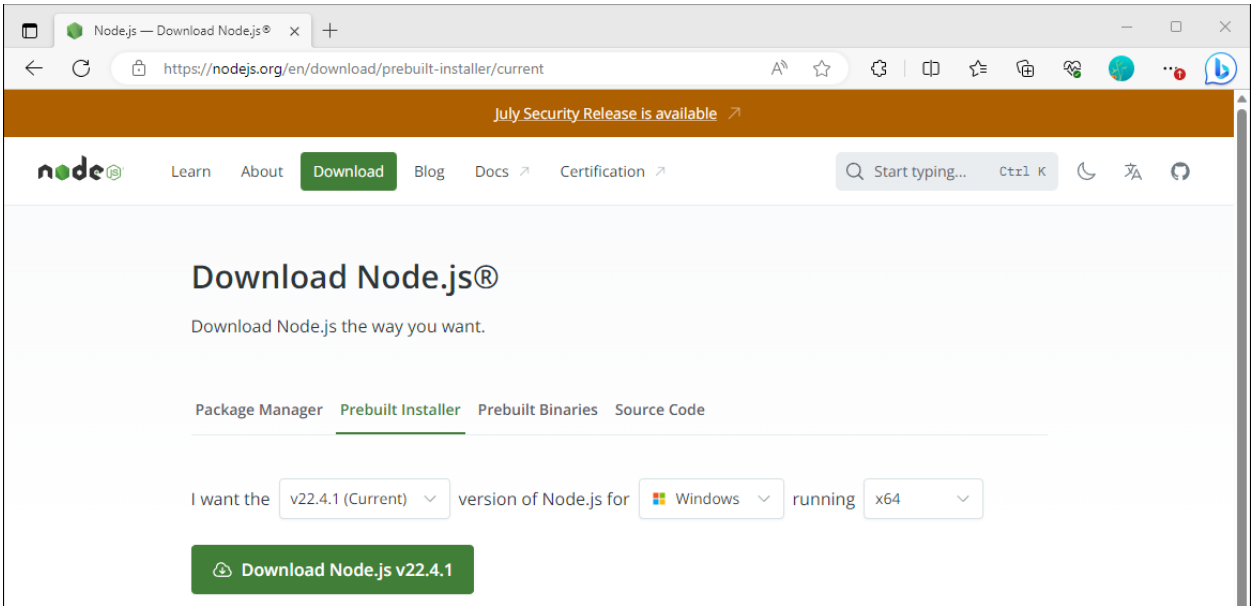
- Current version of React.JS is V18.0.0 (April 2022). Current version of create-react-app is v5.0.1 (April 2022).
- Facebook Software Engineer, Jordan Walke, created React.JS, and it was first used in 2011 for Facebook's Newsfeed feature. create-react-app includes built tools such as webpack, Babel, and ESLint.
- Initial Release to the Public (V0.3.0) was in July 2013.

Installing and running a React JS app

To use React in production, you need **npm** which is included with Node.js. To get an overview of what React is, you can write React code directly in HTML. But in order to use React in production, you need **npm** and Node.js installed.

Node.js® is an open-source, cross-platform JavaScript runtime environment. It runs on the V8 JavaScript engine, and executes JavaScript code outside a web browser. Node.js lets developers use JavaScript to write command line tools and for server-side scripting. The ability to run JavaScript code on the server is often used to generate dynamic web page content before the page is sent to the user's web browser. [1]

To install node.js, visit Node.js (nodejs.org) https://nodejs.org/en



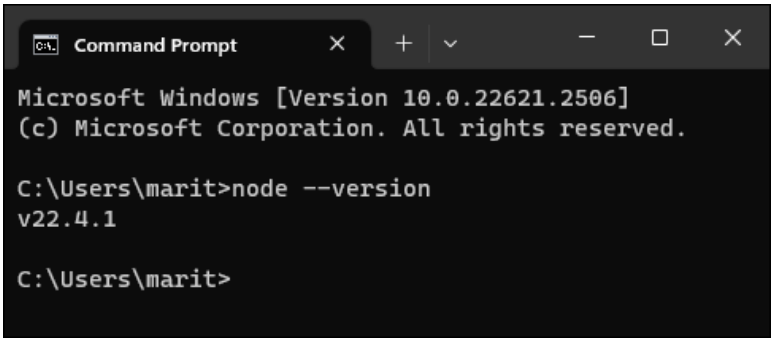
You **can** download the *Recommended For Most Users* version of Node.js. Once you click on the download link, you just need to follow the instructions. **You'll need to have Node.js on your local development machine** (but it's not required on the server).

Once Node.js is installed, we can always check if Node.js is properly installed. To do so, open the ***terminal*** or ***command prompt*** and type one of the following lines:

- `node -v`
- `node --version`

[1] Wikipedia, *Node.js*, Node.js - Wikipedia https://en.wikipedia.org/wiki/Node.js, information retrieved on August 2023

If Node.js is installed, the command prompt should display the version that is installed:



After Node.js is installed, now we are ready to create React apps!

5.2. Setting up a React Environment

To create a React app, you need to install a React app project. To do this, we type the following code in the **command prompt** window:

```
npx create-react-app app_name
```

The command **npx create-react-app app_name** is used to **create a new React project** using the tool *Create React App (CRA)*.

Here's what it does, step by step:

What the command means

- **npx**
  Runs a package from the npm registry *without installing it globally*.
  Here, it temporarily downloads and runs create-react-app.
- **create-react-app**
  A scaffolding tool that sets up a complete React development environment.
- **app_name**
  The name of the new project folder. This will become the directory where your React project is created.

What it actually does

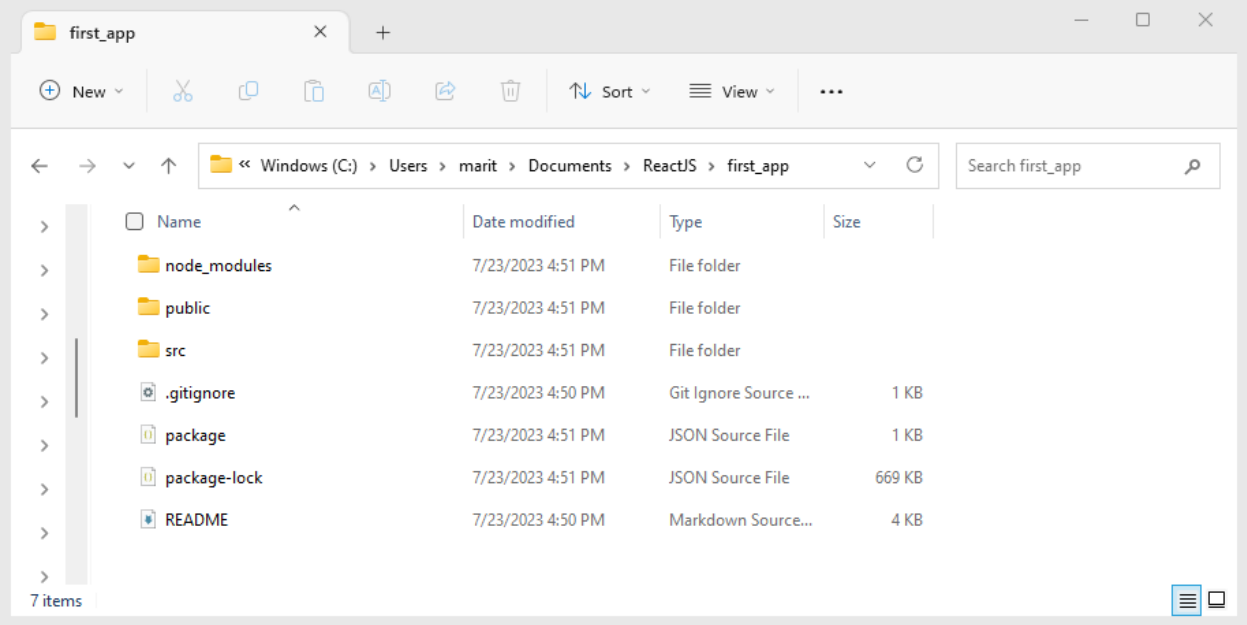
Running this command:

1. **Creates a new folder** called app_name.
2. **Builds a React project structure** inside it.
3. **Installs all necessary dependencies**, such as:
    - React
    - React DOM
    - Webpack
    - Babel
    - Development server and build tools
4. **Configures scripts** like:
    - npm start → run the dev server
    - npm run build → create a production build
    - npm test → run tests

npx create-react-app app_name creates a ready-to-use React application with everything preconfigured so you can start coding immediately.

Resulting folder structure (simplified)

```
app_name
├── node_modules
├── public
├── src
│   ├── App.js
│   ├── index.js
├── package.json
└── README.md
```



No configuration or complicated folder structures, only the files you need to build your app, which is located at the **src** folder.

If you explore inside the React folder, you will see subfolders within the React folder:

- **src** folder is where we put all the source code we write to create the React application.
- **public** folder is where stores static files, like images, music…
- **node_modules** folder contains all of our project dependencies.
- **package.json** records our project dependencies and configures our project.
- **package-lock.json** records the exact version of packages that we installed.
- **README.md** has the instructions on how to use this project.

Running a ReactJS application

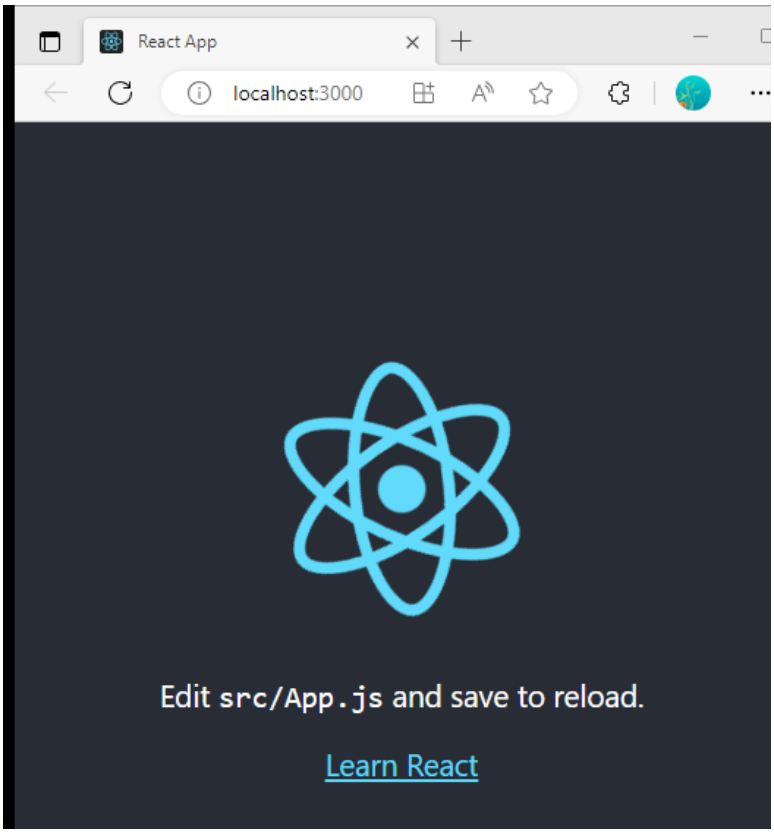
When the React app is created, inside the React app directory, you can run several commands:

- **npm start:** starts the development server.
- **npm run build**: bundles the app into static files for production.
- **npm test**: starts the test runner.
- **npm run eject:** removes this tool and copies build dependencies, configuration files and scripts into the app directory. If you do this, you can't go back!

To run your React app, you have to be at the React app folder:

```
Root_project>cd first_app
```

Then run the React app by typing **npm start** to start the application. When the application runs, it will open a browser at **http://localhost:3000**, where the default React app template will be displayed.



Here are some controls commands and keys of a React application:

- To stop the React App: press **Ctrl + c** at the terminal.
- To start the React App: run **npm start** in the project directory.
- Once the app started, it is hosted at **localhost:3000**

Possible errors that might occur during the running of the React App are:

- Port in use message.
- Localhost:3000 doesn't work.

Creating a React project

To create our own React project, we can start by cleaning the template files from the **src** directory as:

- Clear the content on **App.css** and **index.css** files.
- Modify the **index.js file** as:

```
import React from 'react';
import ReactDOM from 'react-dom/client';
import './index.css';
import App from './App';


const root =
ReactDOM.createRoot(document.getElementById('root'));
root.render(
  <React.StrictMode>
    <App />
  </React.StrictMode>
);
```

- Modify the **App.js** file as:

```
import './App.css';

function App() {
  return (
    <div className="App">

    </div>
  );
}

export default App;
```

5.3. Understanding React Components

React components are the fundamental building blocks of a React application. They allow developers to split the user interface into independent, reusable pieces that can be managed and updated separately.

What Is a React Component?

A React component is a JavaScript function or class that returns JSX (JavaScript XML) to describe what should appear on the screen. Components help organize the UI into modular sections, making code easier to read, maintain, and reuse.

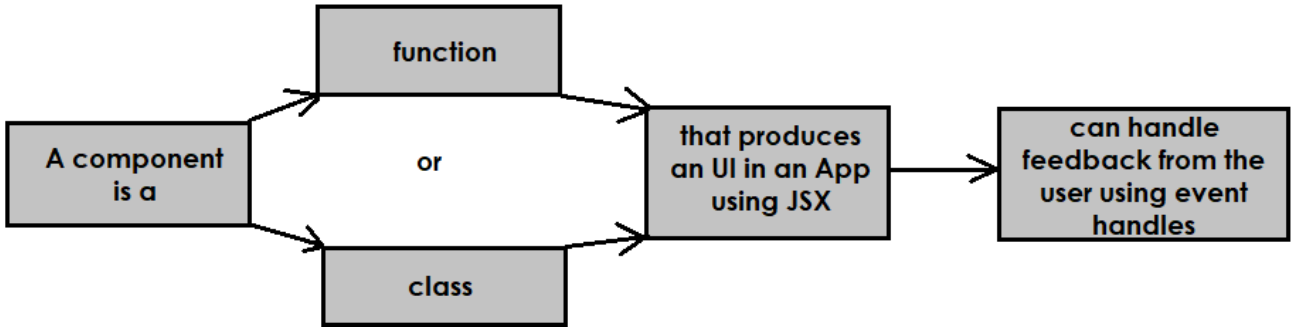
Types of React Components

1. **Functional Components**
   These are simple JavaScript functions that return JSX. They are the most commonly used components in modern React development.

```
function Welcome() {
  return <h1>Hello, World!</h1>;
}
```

2. **Class Components**
   These are ES6 classes that extend `React.Component`. They were commonly used before Hooks were introduced.

```
class Welcome extends React.Component {
  render() {
    return <h1>Hello, World!</h1>;
  }
}
```



5.4. Function components in React

ReactJS Functional components are some of the more common components that will come across while working in React. These are simply JavaScript functions. We can create a functional component in React by writing a JavaScript function. These functions may or may not receive data as parameters. In the functional Components, the return value is the **JavaScript XML, or JavaScript Syntax eXtension, JSX,** code to render to the DOM tree.

We can call the functions in JavaScript by using the name of the function followed by the Parentheses. We can create the app content as a function save in a variable as:

```
import './App.css';
function App() {
  return (
    <div>
          <p>Welcome to React JS</p>
    </div>
  );
}

export default App;
```

In this example, between the `return()` the HTML tag:

```
<div>
     <p>Welcome to React JS</p>
</div>
```

is known as JavaScript XML, JSX.

5.4.1. JSX in React

JSX (JavaScript XML) is a syntax extension for JavaScript that is commonly used with React to describe what the user interface (UI) should look like. It allows developers to write HTML-like code inside JavaScript, making React components easier to create and understand.

What Is JSX?

JSX looks similar to HTML, but it is not exactly the same. It is syntactic sugar for React.createElement() and is compiled into regular JavaScript by tools like Babel.

Example:

```
const element = <h1>Hello, World!</h1>;
```

This JSX code is transformed into:

```
const element = React.createElement("h1", null, "Hello, World!");
```

Why Use JSX?

- Makes UI code more readable and intuitive
- Combines HTML structure and JavaScript logic in one place
- Improves developer productivity
- Reduces complexity when building components

Embedding JavaScript in JSX

You can embed JavaScript expressions inside JSX using curly braces {}.

```
const name = "John";
const element = <h1>Hello, {name}!</h1>;
```

You can also use expressions like:

```
<p>2 + 2 = {2 + 2}</p>
```

JSX Attributes

JSX uses camelCase naming for attributes and slightly different names for some HTML attributes.

Examples:

- class becomes className
- for becomes htmlFor

```
<div className="container">
  <label htmlFor="username">Username</label>
</div>
```

JSX Must Return a Single Parent Element

JSX expressions must have one parent element. You can wrap elements in a or a React Fragment.

```
<>
  <h1>Title</h1>
  <p>Description</p>
</>
```

5.4.2. How to style with JSX

There are different ways that we can style JSX elements. We can:

- Adding custom styling to an element.
- Adding a class to an element.
- JSX can reference to JS variables.

Adding custom styling to an element

Inline styling JSX elements in ReactJS

To style an element with the inline style attribute, the value must be a JavaScript object:

```
<h2 style={{color:"magenta", textAlign:"center"}}>Inline
styling</h2>
```

The differences between HTML inline styling and JSX inline styling are:

- CSS properties written inside curly braces {{}}. The first curly braces is to point to JS and the second curly braces is to reference to a JS object.
- camel-cased property on class properties.

A HTML inline styling looks as:

```
<h2 style="color:magenta, text-align:center">Inline styling</h2>
```

While a JSX inline styling looks as:

```
<h2 style={{color:"magenta", textAlign:"center"}}>Inline
styling</h2>
```

In JSX, JavaScript expressions are written inside curly braces, and since JavaScript objects also use curly braces, the styling in the example above is written inside two sets of **curly braces {{}}.**

Since the inline CSS is written in a JavaScript object, properties with two names, like background-color, must be written with camel case syntax.

Link image in React

We can use the **`require()`** function to link an image in React. We can simply assign the result of **`require()`** to the **`src`** prop without having to store it in a variable.

```
<img src={require('./images/yellowstone.jpg')} />
```

We can also add an image using **`import`** link to import the image to the header app:

```
import yellowstone from './images/yellowstone.jpg';
```

Then when use <img> element to display the image:

```
<img src={yellowstone}/>
```

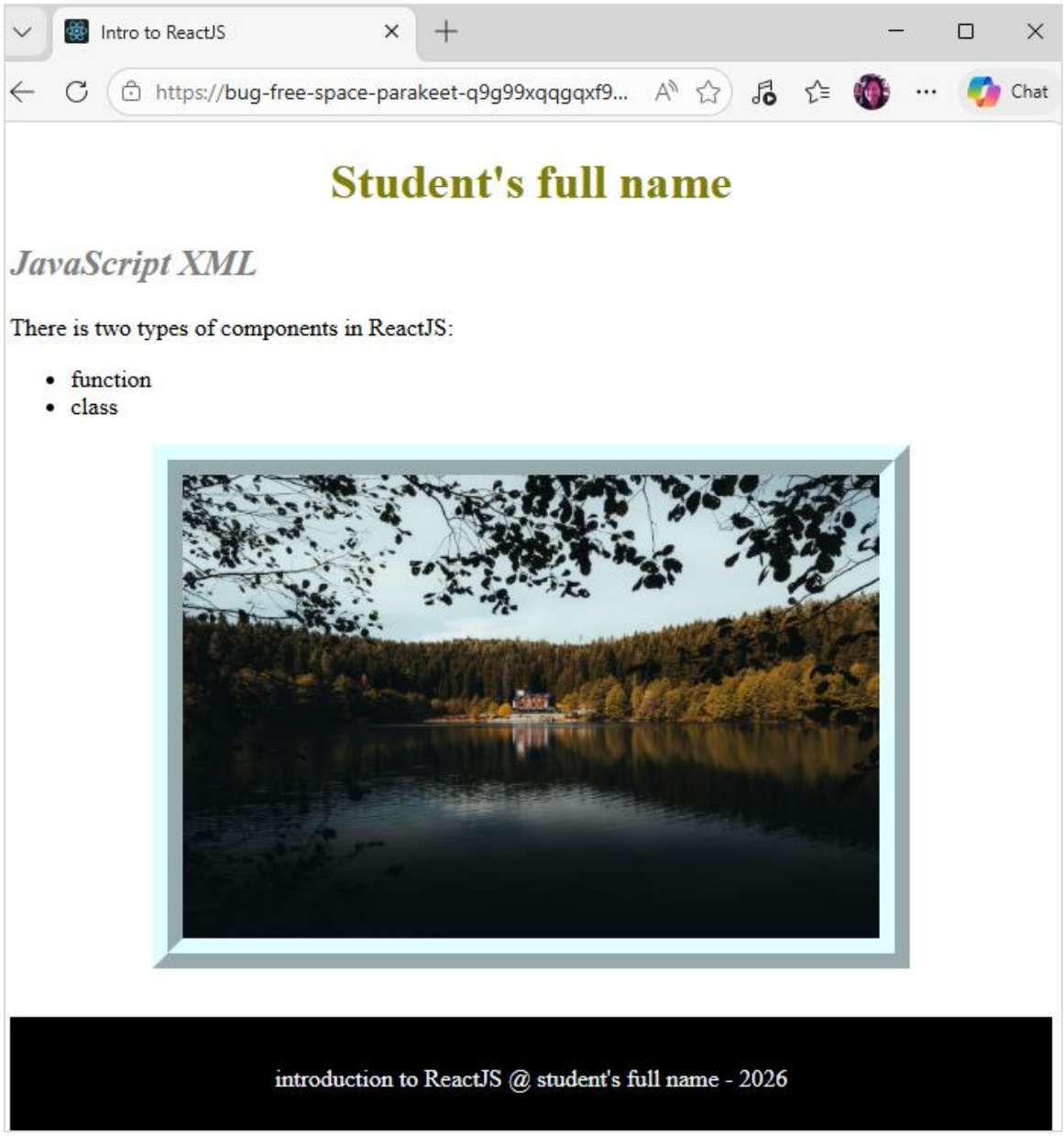
Adding class to an element

To add a class name to a JSX element, we first need to create the class properties:

- in a new CSS file and save the file under **public** folder. Then, link the CSS file in the **index.html <head>** under the **public** folder.
- in the **<head>** of the **index.html** (under the **public** folder) using the **<style>** tag.

Class name is JSX must be in camel-case: so instead of **`class="title`**", we write **`className = "title"`**

Example 1) use different styling to different JSX elements. The output of the JSX looks as:



The complete **App.js** file looks as:

```
import './App.css';
import lake from "./images/lake.jpg"

function App() {
  return (
    <div>
      <h1 style={{color: "olive", textAlign: "center"}}>
          Student's full name</h1>
```

```
      <h2 className='subtitle'>JavaScript XML</h2>
      <p>There is two types of components in ReactJS:</p>
      <ul>
          <li>function</li>
          <li>class</li>
      </ul>
      <figure className='introimg'>
        <img src={lake} />
      </figure>
      <footer className='footer'>
        introduction to ReactJS @ student's full name - 2026
      </footer>
    </div>
  );
}

export default App;
```

5.4.3. JS variables in JSX

Before **ES6**, there was only one way to declare variables: using the var keyword. If you declared a variable without var, it was automatically added to the global object. However, in **strict mode**, using an undeclared variable would throw an error.

With **ES6**, we now have three ways to declare variables: var, let, and const.

var

- If declared outside a function, it belongs to the **global scope**.
- If declared inside a function, it belongs to that **function scope**.
- If declared inside a block (like a for loop), it is still accessible **outside** that block.
- var has **function scope**, not block scope.

let

- let is the block-scoped version of var.
- If declared inside a block (such as a for loop), it is only accessible within that block.
- let has block scope.

const

- const is also block-scoped.
- Once assigned, its value cannot be reassigned.

- However, const does not create a constant value — it creates a constant reference to a value.

Because of this, you **cannot**:

- Reassign a constant variable
- Reassign a constant array
- Reassign a constant object

But you **can**:

- Modify elements inside a constant array
- Modify properties of a constant object

5.4.4. Using Variables in React JSX

In **React JS**, curly braces {} are used to insert JavaScript variables or expressions inside JSX.

Example 2: Creating a constant variable and displaying it inside a

```
const divText = "Hello, welcome to React!";

function App() {
  return (
    <div>
      {divText}
    </div>
  );
}

export default App;
```

We can also return a function call inside a JSX component.

In React, JSX allows you to use curly braces {} to execute JavaScript expressions, including function calls, and display their returned values.

Example 3: Returning a function inside a JSX component

```
function App() {

  const getMessage = () => {
    return "Welcome to React!";
  };
```

```
  return (
    <div>
      {getMessage()}
    </div>
  );
}

export default App;
```

Explanation:

- `getMessage` is a function that returns a string.
- Inside JSX, `{getMessage()}` calls the function.
- The returned value is rendered inside the `<div>`.

You can use this approach to dynamically generate content, calculate values, or conditionally render elements inside your React components.

5.4.5. React App usings semantic UI styling

Now that we have learned the basic structure of React JS and how to create a React app using JSX components, the next step is to build a React application using **Semantic UI** for styling.

First, open your terminal and stop the currently running React app by pressing **Ctrl + C**. Then, navigate back one directory from your previous React project folder using:

- `cd ..` (Mac or Linux)
- `.. cd` (Windows)

Next, we need to include Semantic UI in our project. Visit the following website to get the Semantic UI CDN link:

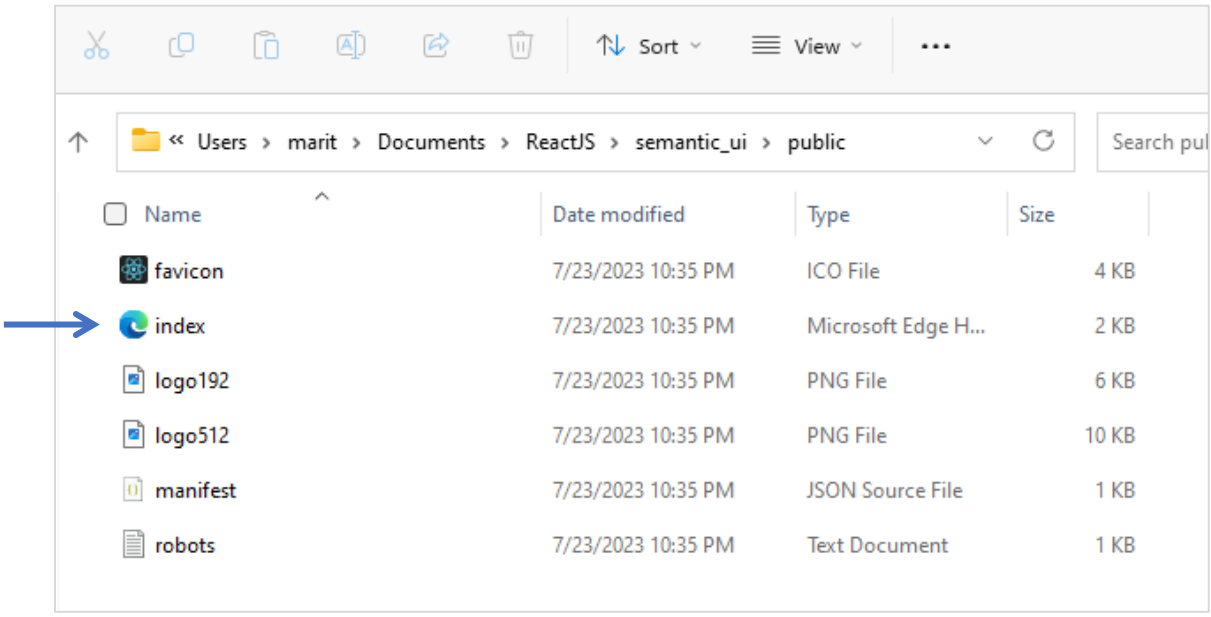
👉 https://cdnjs.com/libraries/semantic-ui

Copy the CSS CDN link and paste it into the index.html file located inside the **public** folder of your React project.

It should be added inside the <head> section like this:

```
<link rel="stylesheet" href="YOUR_SEMANTIC_UI_CDN_LINK_HERE" />
```

After saving the file, restart your React app, and you will be able to use Semantic UI classes inside your components.

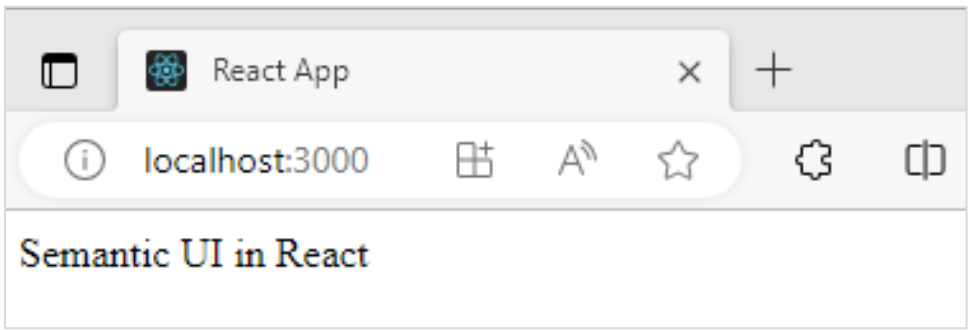


Once in the **index.html** file, we can add the semantic UI link within the `<head>` tag as:

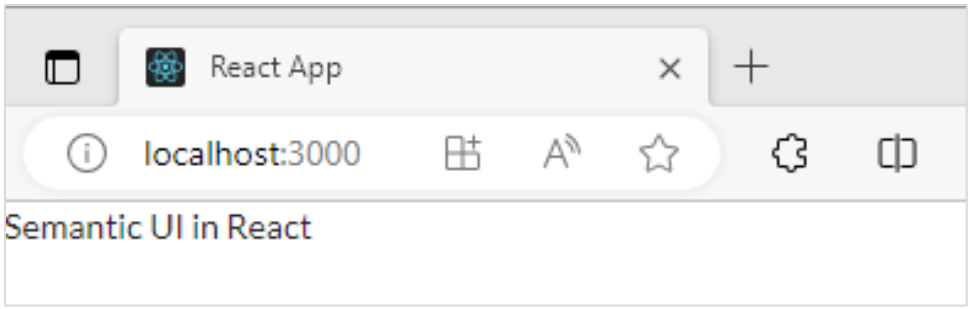
```
<link rel="stylesheet"
href="https://cdnjs.cloudflare.com/ajax/libs/semantic-
ui/2.5.0/semantic.min.css">
```

You might use a different version of semantic UI. The browser should look as:

Without semantic UI styling



With semantic UI styling



Visit [Get Started - Semantic UI React (semantic-ui.com) https://react.semantic-ui.com/usage/](https://react.semantic-ui.com/usage/) for more information on how to use the semantic UI styling class name.

Exercise 1): Styling a React App with the Assistance of AI

Objective

Learn how to style a React application using different styling methods (CSS, Inline Styles, and CSS Frameworks) while using AI tools to assist with generating and improving your code.

Scenario

You are building a simple **Profile Card** React application. Your task is to style it in three different ways with the help of AI.

Part 1: Create the Basic React Component

Create a new React app and build the following component:

```
function ProfileCard() {
  return (
    <div>
      <h2>John Doe</h2>
      <p>Frontend Developer</p>
      <button>Follow</button>
    </div>
  );
}

export default ProfileCard;
```

Render it inside your App.js.

Part 2: Styling with Plain CSS (AI Assisted)

Task:

1. Create a ProfileCard.css file.
2. Ask AI to:
    - Center the card on the page
    - Add padding and margin
    - Add border radius
    - Add box shadow
    - Style the button with hover effects

Example AI Prompt:

"*Generate modern card styling CSS for a React profile card with soft shadow, rounded corners, centered layout, and a blue hover button*."

Import the CSS file into your component:

```
import "./ProfileCard.css";
```

Add class names to your JSX and apply the styles.

Part 3: Styling with Inline Styles (AI Assisted)

Now remove the CSS file and instead:

1. Ask AI to convert your CSS into inline styling.
2. Create a const styles = {} object.
3. Apply styles using:

```
<div style={styles.card}>
```

Example AI Prompt:

"Convert this CSS card design into React inline style object format."

Part 4: Styling with a CSS Framework (Optional Bonus)

Install a framework like Bootstrap or use a CDN.

Example:

```
npm install bootstrap
```

Import it in **`index.js`**:

```
import 'bootstrap/dist/css/bootstrap.min.css';
```

Now ask AI:

"Convert my profile card into a Bootstrap styled card component."

Part 5: AI Enhancement Challenge

Use AI to:

- Add gradient background
- Add animation on hover
- Make the card responsive
- Improve typography
- Add dark mode styling

Reflection Questions

1. What are the differences between CSS files and inline styling?
2. When should you use a CSS framework?
3. How did AI improve your styling workflow?
4. What should you verify when using AI-generated code?

5.5. React Routing: Navigation in Single Page Applications (SPA)

Introduction to React Routing

React Routing allows you to create navigation between different components in a React application without reloading the page. Since React is a **Single Page Application (SPA)** framework, routing enables dynamic content updates while keeping the user on the same HTML page.

The most used library for routing in React is **React Router**.

Why Do We Need Routing?

In traditional websites:

- Each link loads a new HTML page from the server.

In React:

- We load one main HTML file.
- Different components are displayed based on the URL.
- The page does not refresh.

Routing makes this possible.

Installing React Router

To use routing in your React project, install React Router:

```
npm install react-router-dom
```

Routing is the capacity to show different pages to the user. That means the user can move between different parts of an application by entering a URL or clicking on an element.

As you may already know, by default, React comes without routing. And to enable it in our project, we need to add a library named react-router.

To install the react-router, you will have to run the following command in the Terminal under the project folder. For example, we can create first a react project named `reactrouting`

```
npx create-react-app reactrouting
```

After the react app is created, we navigate into the react app project folder by typing in the Terminal

```
cd reactrouting
```

At the project folder, we install the react router by typing in the Terminal:

```
yarn add react-router-dom
```

or

```
npm install react-router-dom
```

5.5.1. Setting up the router

To enable routing in our React app we need to set up the files first.

In the **App.js** file, enter the following:

```
import React from 'react'

function App() {
  return (
    <div>
        <h1 className='title'>React Routing by ______ </h1>
    </div>
  );
}

export default App;
```

Let's create a new directory, folder, named components within the src folder to keep all other files. Inside the components folder, we are doing to create four js files, (about.js, contact.js, home.js, navbars.js) which are the files that we are going to use to route:

```
Reactrouting/
     |_______ App.js
     |_______ index.js
     |_______ index.css
     |_______ components/
                    |_______about.js
                    |_______contact.js
                    |_______home.js
                    |_______navbars.js
```

We are going to create a template for contact.js, home.js, and navbars.js files as:

Contact.js

```
import React from "react";

const Contact = function(){
    return(<>
        <header className="header">
            <h1 className="headertitle">Contact Us</h1>
        </header>

    </>
    )
}

export default Contact
```

home.js

```
import React from "react";
import computerimg from '../images/computer.jpg'

const Home = function(){
    return(<>
        <header className="header">
            <h1 className="headertitle">Homepage</h1>
        </header>
        <main className="maincontainer">
            <div>
                <img src={computerimg}/>
            </div>
        </main>
    </>
    )
}

export default Home
```

5.5.2. Link

The Link component is used to create navigation links in your application. It is similar to an HTML <a> tag but provides the benefits of client-side routing, which means the navigation is handled by React Router without causing a full page reload.

- **Outlet:** Used for rendering nested routes in a parent route's component.
- **Link:** Used for creating navigation links that allow for client-side routing.

Anytime we link to an internal path, we will use <Link> instead of <a href="">.
The "Outlet" is a shared component that inserts common content on all pages, such as a navigation menu.

Now, we need to adjust the pages and components. We can start will the navbars.js file. In React, Outlet and Link are concepts related to React Router, which is a popular library for handling routing in React applications.

Outlet

The Outlet component is used in nested routes to render child routes. When defining a parent route that has nested routes, the Outlet serves as a placeholder where the nested route components will be rendered. It allows you to build complex layouts with nested UI components.

Example usage:

```
import { Outlet } from 'react-router-dom';
function ParentComponent() {
  return (
    <div>
      <h1>Parent Component</h1>
      <Outlet />
          {/* This is where the child routes will be rendered */}
    </div>
  );

}
```

navbars.js

```
import React from "react";
import { Link } from "react-router-dom";

const Navbars = function(){
    return(<>
       <header className="header">
          <nav className="navbar">
             <div>
                    <a href=https://www.qcc.cuny.edu/
                    target="blank" className="link">QCC</a>
              </div>
              <div className="btngroup">
                 <button><Link to='/contact' className="link">
                    Contact Us</Link></button>
                 <button><Link to='/' className="link">
                    Home</Link></button>
              </div>
            </nav>
        </header>
    </>
    )
}

export default Navbars
```

In the navbars.js file, we want to be able to link to each js file when we click on the tab. For this, we are running our web app in the localhost:3000, we need to add React router in my application. After react-router is installed, we are import React router to our App.js file as:

```
import {BrowserRouter, Routes, Route } from "react-router-dom"
```

We can also style the app through the **App.css** file

```
*{box-sizing: border-box;}
img{width: 100%; display: block;}
.header{
    color: darkblue;
    text-align: center;
}

/*home.js file*/
.maincontainer{
    width: 900px;
    margin: 2rem auto;
    overflow: hidden;
    border-radius: 1rem;
}

/* navbars.js */
.navbar{
    background-color: darkorange;
    padding: 1rem;
    display: flex;
}
.navbar div:nth-child(1){
    width: 30%;
}
.navbar div:nth-child(2){
    width: 70%;
}
.navbar div:nth-child(2) button{
    float: right;
    background-color: rgba(0,0,0,0);
    border: none;
}
```

```
.link{
    text-decoration: none;
    color:aliceblue;
    font-size: 1.2rem;
    padding: 0.5rem 2rem;
}
.link:hover{
    color: darkblue;
}

/*app.js*/
.title{ color: darkblue; text-align: center;}
```

Next, we can set up routing to the home.js, about.js, and contact.js files using BrowserRouter, Routes, and Route.

First, we wrap the entire application inside <BrowserRouter>. Then, we define our <Routes> container. An application can contain multiple routes, but in this basic example, we will use a simple structure with a few pages.

After that, we use <Route> to map each path to its corresponding component, which can then be accessed through the navigation bar (Navbar) as follows:

App.js

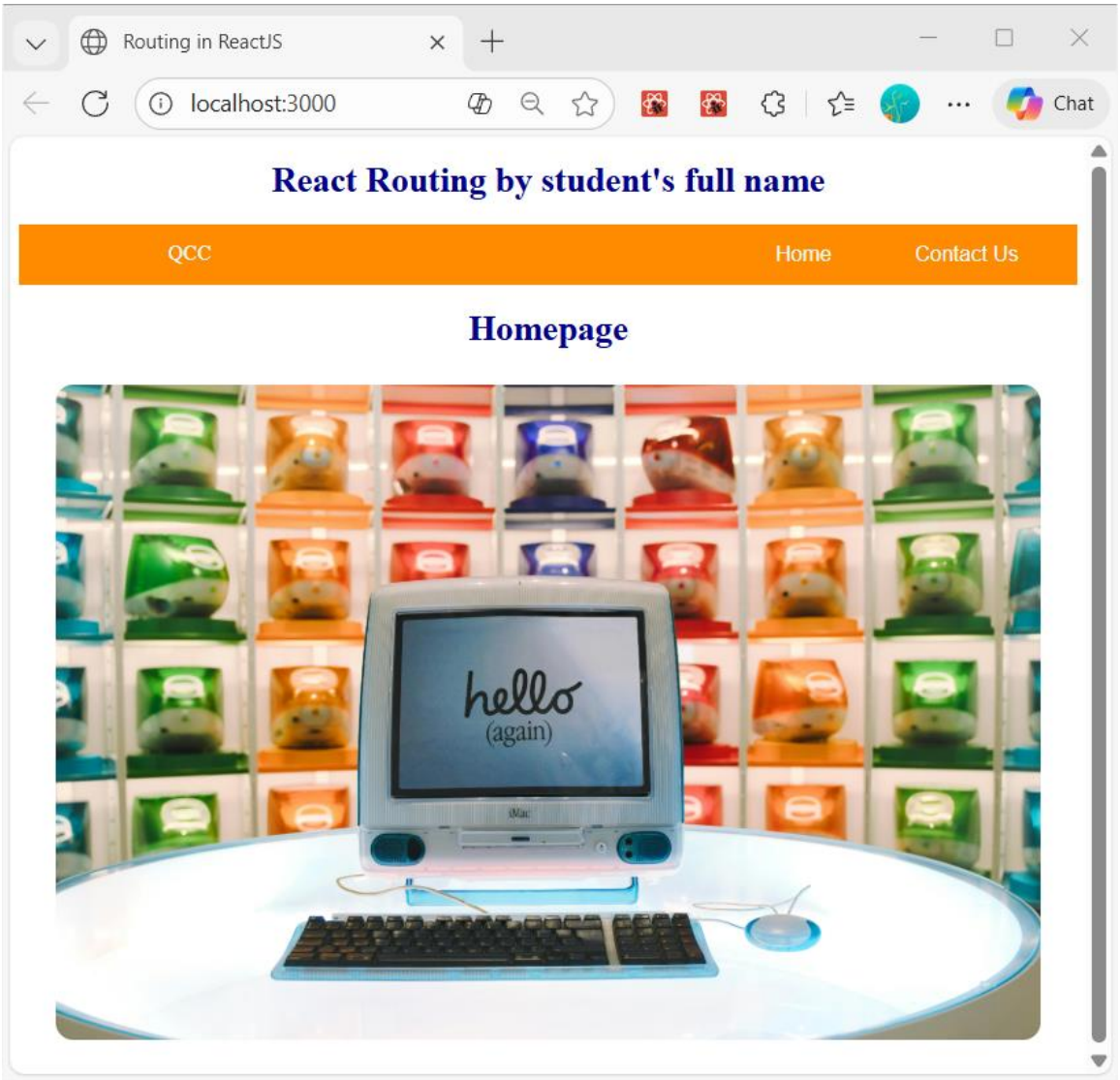
```
import './App.css';
import React from 'react';
import { BrowserRouter, Route, Routes } from 'react-router-dom';
import Navbars from './components/navbars';
import Home from './components/home';
import Contact from './components/contact';

function App() {
 return (
  <>
  <h1 className='title'>React Routing by student's full name</h1>
      <BrowserRouter>
        <Navbars />
        <Routes>
          <Route path='/' element={<Home />} />
```

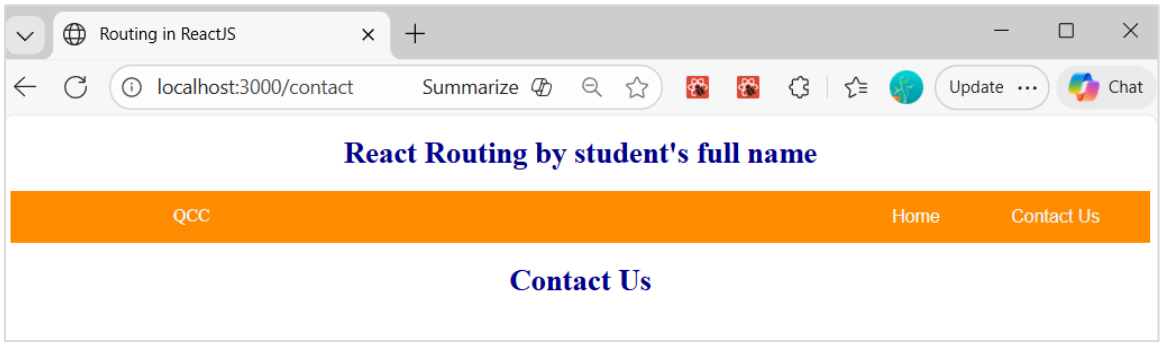
```
          <Route path='/contact' element={<Contact />} />
        </Routes>
      </BrowserRouter>
    </>
  );
}

export default App;
```

The browser window will show:



To test the routing in the webpage, we can click on the **Contact Us** tab and it should route to the Contact Us webpage:



Exercise 2: Practice React Routing with the Help of an AI Assistant

Objective

Learn how to implement routing in a React application using **React Router**, while practicing how to effectively use AI to guide, generate, and improve your code.

Project Scenario: Student Portal App

You are building a simple **Student Portal** with multiple pages:

- Home
- Students
- Student Details
- About
- 404 Not Found Page

You will use an AI assistant to help you implement each step.

Part 1: Setup React Router

Step 1: Install React Router

Use the terminal:

npm install react-router-dom

AI Task

Ask AI: “Show me how to properly configure BrowserRouter in a React application.”

Implement the suggested setup in your App.js.

Part 2: Create Basic Routes

Create the following components:

- Home.js
- Students.js
- About.js

Each should return a simple <h2> title.

AI Task

Ask AI: "Generate routing code to connect Home, Students, and About pages using React Router v6."

Your routes should look like:

```
/ → Home
/students → Students
/about → About
```

Part 3: Add Navigation Links

Use the Link component instead of <a> tags.

AI Task

Ask AI: "Show me how to create a navigation bar using Link in React Router."

Create a simple navigation menu at the top of your app.

Part 4: Dynamic Route (URL Parameters)

Now create a **Student Details** page.

Task:

- Create StudentDetails.js
- The route should look like:
  `/students/:id`

AI Task

Ask AI: "How do I use useParams in React Router to get the student ID from the URL?"

Display the student ID inside the component.

Example:

Student ID: 101

Part 5: 404 Not Found Page

Create a NotFound.js component.

AI Task

Ask AI: “How do I create a wildcard route in React Router v6 for a 404 page?”

Use:

path="*"

Part 6: Programmatic Navigation

Add a button on the Home page that navigates to the Students page when clicked.

AI Task

Ask AI: “How do I navigate programmatically using useNavigate in React Router?”

Reflection Questions

1. What is the purpose of BrowserRouter?
2. Why should we use Link instead of <a>?
3. What are URL parameters used for?
4. How does useNavigate improve user interaction?
5. What did AI help you understand better?

5.6. Properties, props, system in React JS

props stands for **'properties'** and are arguments passed into React components via HTML attributes. **props** are also pass data from one component (parent) to another (child) component as parameters. Props purpose is to customize or configure a child component. Conceptually, components are like JavaScript functions. They accept arbitrary inputs (called "props") and return React elements describing what should appear on the screen.

The simplest way to define a component using props is to write a JavaScript function and pass props as an event:

```
function Welcome(props) {
  return <h2>Welcome to React props {props.name}!</h2>;
}
```

This function accepts a single "props" object argument with data and returns a React element.

React **props** are like function arguments in JavaScript and attributes in HTML. To send props into a component, use the same syntax as HTML attributes:

```
import React from "react"

const App = function(props){

     return(
        <>
           return <h2>Welcome to React props {props.name}!</h2>
        </>
     )
}

const myElement = <App name ="Martha" />

export default User_feedback;
```

React Props can help programmers to reuse and nest component.

- Component nesting is when a component can be shown inside of another component.
- Component reusability helps programmers to reduce the code lines of repeating component.
- Programmers should be able to configure a component when it is created.

How to use props to create reusable components?

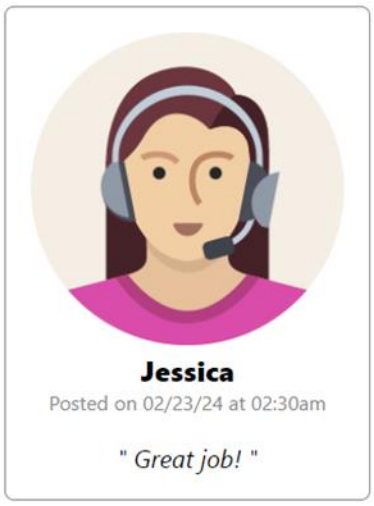
To use Props to create reusable components by following steps:

1. Identify the JSX that appears to be duplicated.
2. Think of a descriptive name for reusable component.
3. Create a new file for reusable components.
4. Create a new component in the new file.
5. Make the new component configurable by using **props** system.

Example 3) Generate multiple card components that use the same HTML structure and accept props, matching the functionality in the app shown below.



For this app, we can see that the JSX element being duplicated is a card with an image at the top and three text elements at the bottom.

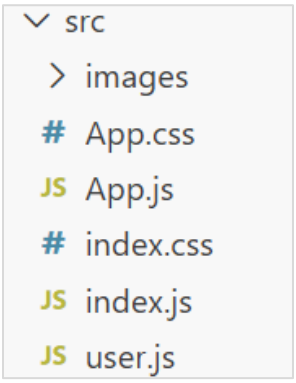


Step 1) Setting up the React environment

For first-time React users, start by installing the React app generator through the terminal:

```
npx create-react-app react_props
```

After that, create a new .js file named **user.js** to define the UI for the user card. Place the user.js file inside the **src** folder.



In the user.js file, we will create the structure of the main JSX component. This component is defined inside a function called **User**. Once the component structure is complete, we export the User function so it can be used in **App.js** with the following import statement: `import User from './user';`

user.js

```
import React from "react";
import avataruser1 from './images/avatar_batman.png'
import './App.css'

const User = function(){
    return(
      <>
          <section className="card">
             <a><img src={ avataruser1} /> </a>
             <div className="content">
                 <a className="author">Batman</a>
             </div>
             <div className="metadata">
                 <p>Posted on <span className="date">TODAY
                    </span></p>
             </div>
             <div className="comment">
                 <p><i>"testing message"</i></p>
             </div>
           </section>
        </>
    )
}

export default User
```

The styling of the react app, ***App.css,*** will be as follows:

```
/* CARD - COMMENTS.JS */
.card{
  padding: 1em;
  width: 90%;
  text-align: center;
  margin: 1em auto;
  border-radius: 10px;
  transition: ease-in-out 100ms;
}
```

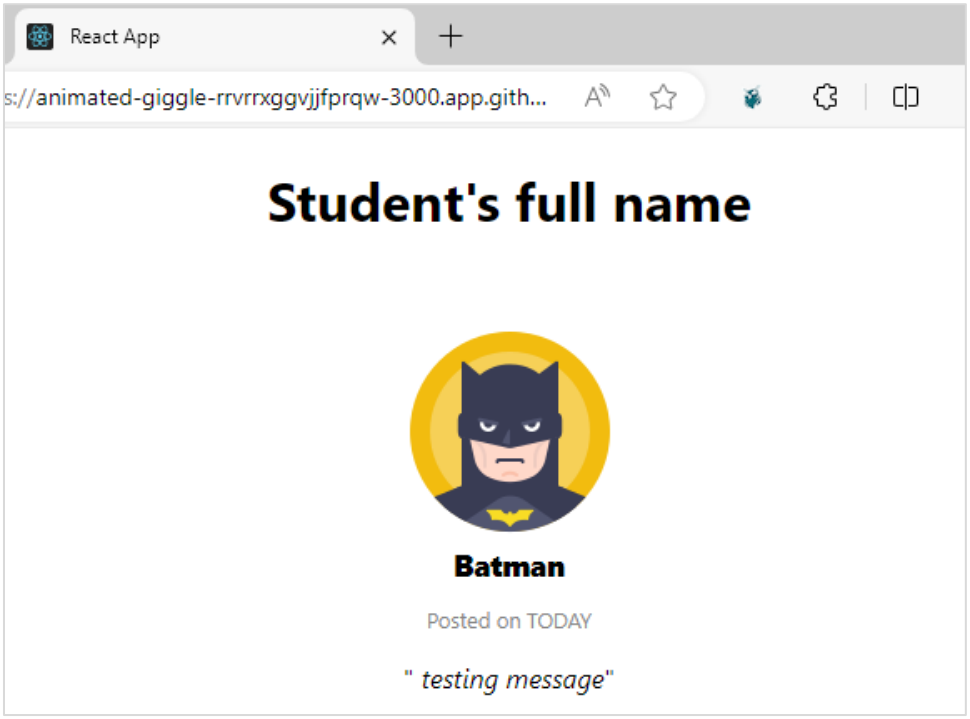
```
.content{
  font-size: 1.1em; font-weight: 900;
}
.metadata{
  font-size:0.8em; color: #888;
}
.container{
  display: flex;
  justify-content: center;
  flex-wrap: nowrap;
}
```

Now, let us create the main JSX component in the **App.js** file.

```
import React from 'react';
import ReactDOM from 'react-dom/client';
// import files
import './App.css';
import User from './comments'

const App = function(){
  return(
    <>
      <h1 style={{textAlign:"center"}}>Student's full name</h1>
      <section className='container'>
        {/* user 1 */}
       <User />
      </section>
    </>
  )
}
const root =
ReactDOM.createRoot(document.getElementById('root'));
root.render(
  <React.StrictMode>
    <App />
  </React.StrictMode>
);
```

Running the server, **`npm start`**, should show the following:



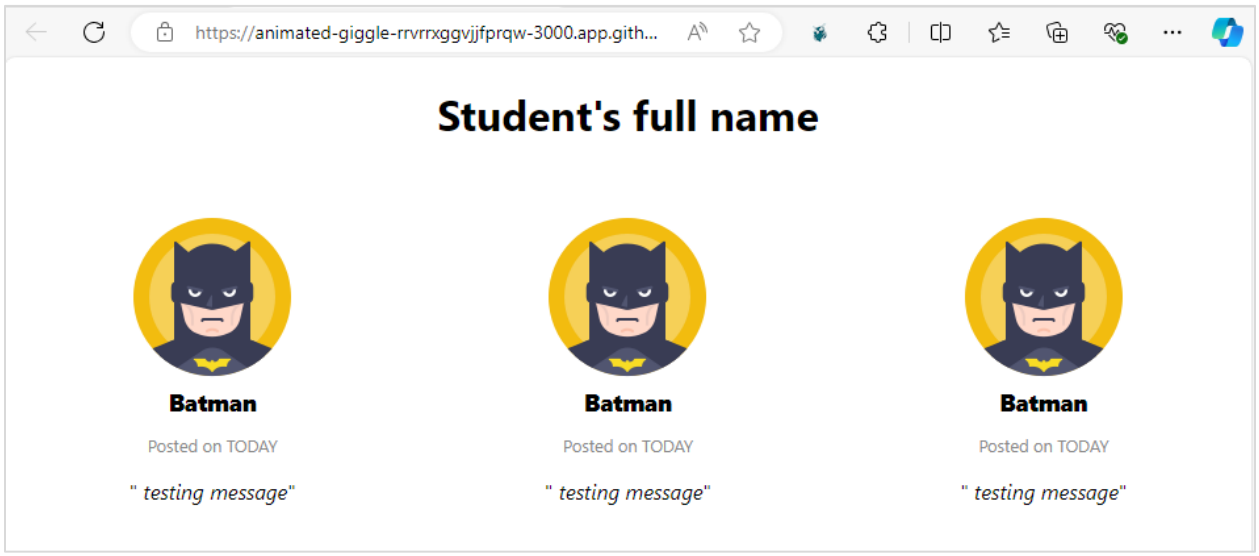
If you want to use the same JSX structure <User />, we call <User /> again in the ***App.js*** file:

```
import React from 'react';
import ReactDOM from 'react-dom/client';
// import files
import './index.css';
import User from './comments'

const App = function () {
  return (
    <>
      <h1 style={{textAlign: "center"}}>Student's full name</h1>
      <section className='container'>
        {/* user 1 */}
        <User />
        {/* user 2 */}
        <User />
```

```
        {/* user 3 */}
        <User />
      </section>
    </>
  ) }
```

The internet browser, *localhost:3000,* should look as:



After the main JSX component is created, we move to create a child component in the **App.js** file. In the component, we will create properties called *name, msg,* and *image* and assign them to three different components, users:

App.js

```
import React from 'react';
import ReactDOM from 'react-dom/client';

// import all images
import avataruser1 from './images/batman.png';
import avataruser2 from './images/businessman.png';
import avataruser3 from './images/lady.png';

// import files
import './index.css';
import User from './comments'

const App = function(){
```

```
  return(
   <>
    <h1 style={{textAlign:"center"}}>Student's full name</h1>
    <section className='container'>

    {/* user 1 */}
    <User name="Batman" image={avataruser1} date="11/12/24"
     msg="I am Batman"/>

    {/* user 2 */}
    <User name="Peter" image={avataruser2} date="10/03/24"
     msg="I am late!"/>

     {/* user 3 */}
    <User name="Lady" image={avataruser3} date="08/20/24"
     msg="Get more rest"/>

 </section>
 </>
 )
}

const root =
ReactDOM.createRoot(document.getElementById('root'));
root.render(
  <React.StrictMode>
    <App />
  </React.StrictMode>
);
```

Now we are going to pass the properties **name**, **msg**, **date**, and **image** from the index.js file into the component in the comments.js file.

user.js

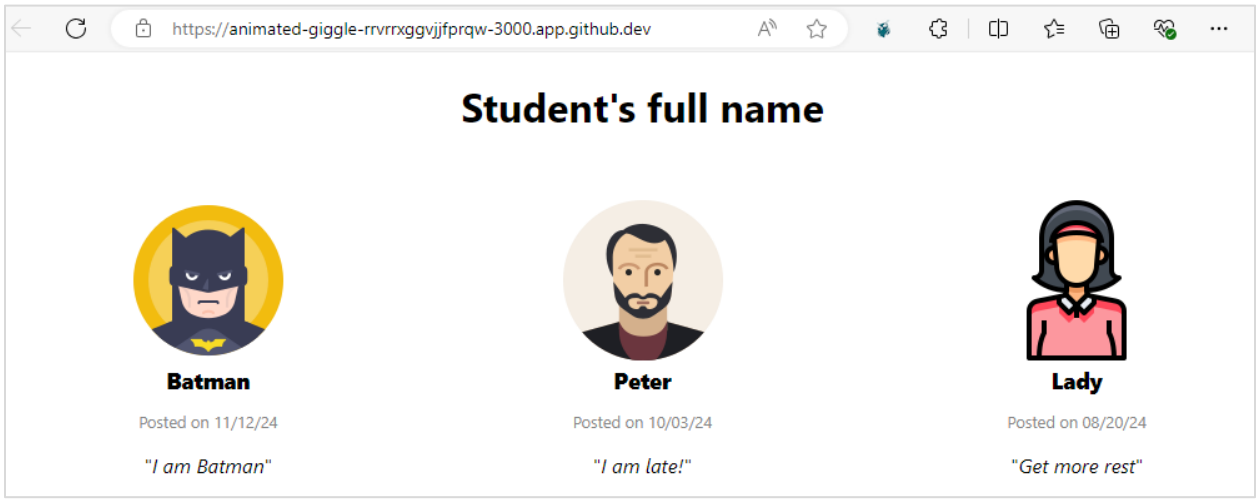
```
import React from "react";

const User = function(props){
   return(
      <>
         <section className="card">
            <a> <img src={props.image}/> </a>
```

```
            <div className="content">
                 <a className="author">{props.name}</a>
            </div>
            <div className="metadata">
               <p>Posted on <span  className="date">{props.date}
                    </span></p>
            </div>
            <div className="comment">
               <p>"<i>{props.msg}</i>"</p>
            </div>
          </section>
        </>
    )
}

export default User;
```

In the browser at localhost:3000, you should see each prop—**name**, **msg**, **date**, and **image**—reflected in the updated UI.



5.6.1. Adding props children in React

We're almost done! The last component we need to add is a link that allows the user to submit a comment. To implement this, we will create a component that opens a window for adding a comment to an existing card.

Create a new file called **user_feedback.js** in the src folder to set up the feedback component. We will create and test the feedback component first.

User_feedback.js

```
import React from "react";
import './App.css'
const User_feedback = function(){
    return(
       <>
          <div className="feedbackcontainer">
            <section className="feedbackcard">
               <div className="content">
                    <p>Name of user</p>
               </div>
               <div className="description">Some feedback</div>
               <div className="cardfooter">
                  <p className="addicon"><span>&#10011;</span>
                    Add feedback
                  </p>
                </div>
            </section>
          </div>
        </>
    )
}
export default User_feedback
```

We can update the style of the feedback card in the 'App.css' file as:

```
/* USER_FEEDBACK.JS*/
.feedbackcard{
  width: 300px;
  border: #888 solid 1px;
  border-radius: 10px;
  text-align: center;
  overflow: hidden; }
.cardfooter{
  background-color: #888;
  padding: 0.2em;
  color: aliceblue;
}
.feedbackcontainer:hover{
  box-shadow: 0px 0px 10px 1px gray;
  transform: scale(1.01);
```

```
  overflow: hidden;
  border-radius: 10px;
}
.feedbackcontainer{
  margin: 1%;
}
```

We import the u*ser_feedback* to the App.js file using the *import* keyword.

App.js

```
import React from 'react';
import ReactDOM from 'react-dom/client';

// import all images
import avataruser1 from './images/batman.png';
import avataruser2 from './images/businessman.png';
import avataruser3 from './images/lady.png';

// import files
import User_feedback from './user_feedback';
import './App.css';
import User from './user'

const App = function(){
  return(
   <>
    <h1 style={{textAlign:"center"}}>Student's full name</h1>
    <section className='container'>
          {/* user 1 */}
          <User name="Batman" image={avataruser1} date="11/12/24"
          msg="I am Batman"/>

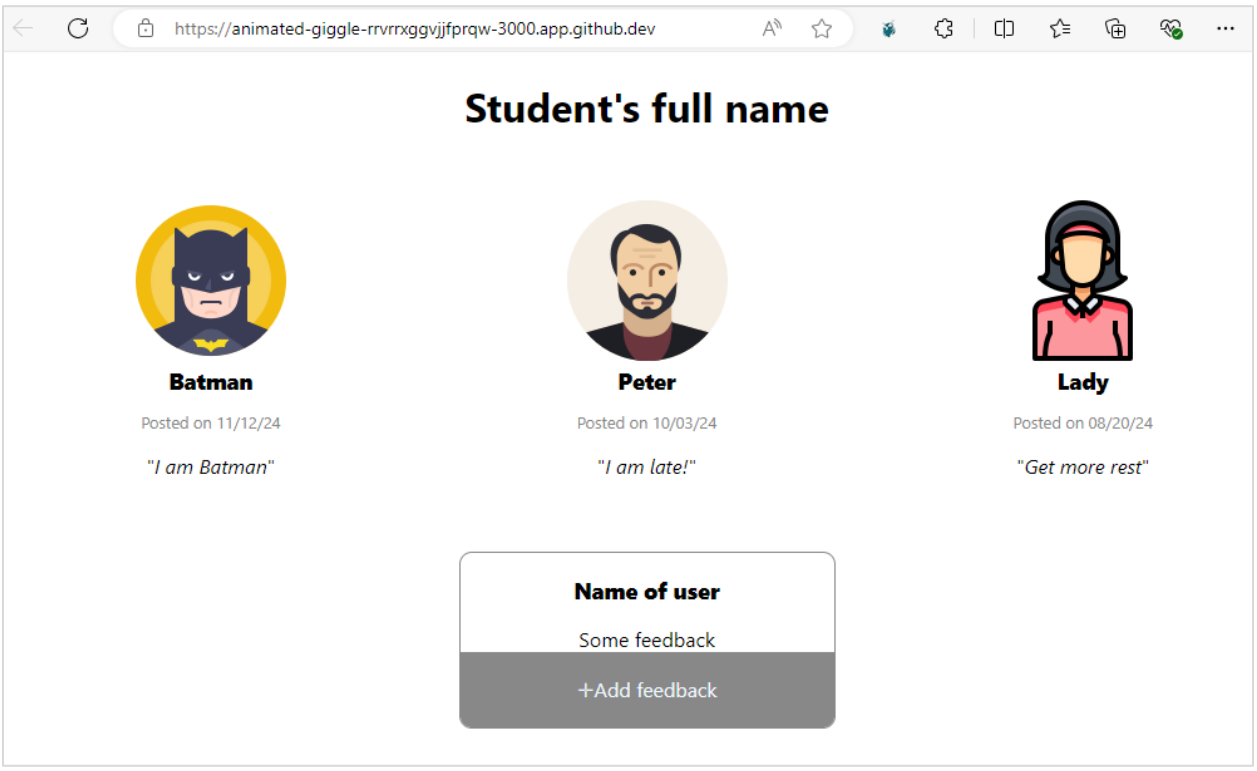
          {/* user 2 */}
          <User name="Peter" image={avataruser2} date="10/03/24"
          msg="I am late!"/>

          {/* user 3 */}
          <User name="Lady" image={avataruser3} date="08/20/24"
          msg="Get more rest"/>
   </section>
```

```
    <section className='container'>
      <User_feedback></User_feedback>
    </section>
  </>
  )
}

const root =
ReactDOM.createRoot(document.getElementById('root'));
root.render(
  <React.StrictMode>
    <App />
  </React.StrictMode>
);
```

The browser should look as:



We can also pass a component as a **child component** using props. To do this, in the index.js file, we will use the <User> component as a child of the <User_feedback> component:

App.js

```
import React from 'react';
import ReactDOM from 'react-dom/client';

// import all images
import avataruser1 from './images/batman.png';
import avataruser2 from './images/businessman.png';
import avataruser3 from './images/lady.png';

// import files
import User_feedback from './user_feedback'
import './App.css'
import User from './user'

const App = function(){
  return(
   <>
    <h1 style={{textAlign:"center"}}>Student's full name</h1>

    <section className='container'>

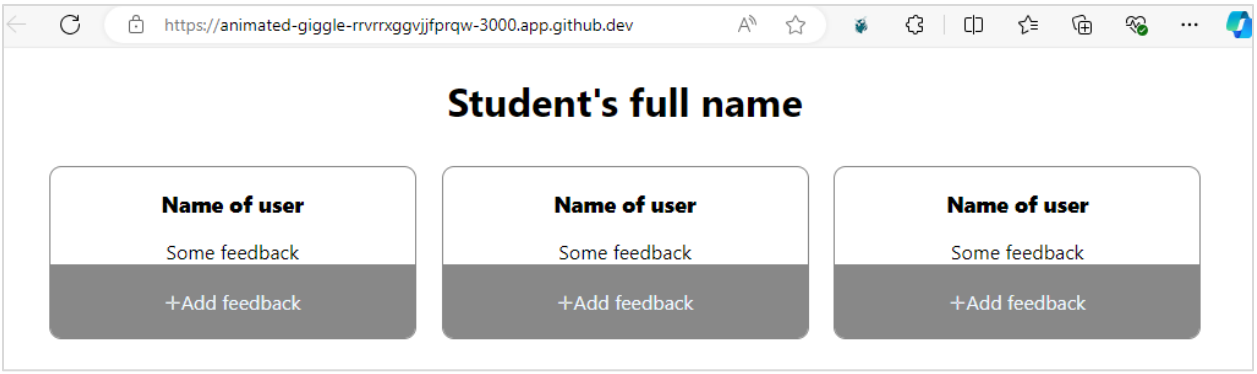
          {/* user 1 */}
          <User_feedback><User name="Batman" image={avataruser1}
          date="11/12/24" msg="I am Batman"/></User_feedback>

          {/* user 2 */}
          <User_feedback><User name="Peter" image={avataruser2}
          date="10/03/24" msg="I am late!"/></User_feedback>

          {/* user 3 */}
          <User_feedback><User name="Lady" image={avataruser3}
          date="08/20/24" msg="Get more rest"/></User_feedback>

   </section>
 </>
 )
}
```

The browser will look as:



After that, we need to use **props** in the User_feedback.js file to pass the child components into the feedback cards.

User_feedback.js

```
import React from "react";
import './index.css'

const User_feedback = function(props){
    return(
        <>
          <div className="feedbackcontainer">
            <section className="feedbackcard">
              <div className="content">
                    <p>{props.username}</p></div>
              <div className="description">{props.children}</div>
              <div className="cardfooter">
                 <p className="addicon"><span>&#10011;</span>
                    Add feedback</p>
              </div>
            </section>
          </div>
        </>
    )
}
export default User_feedback
```

Now, we need to pass the 'props' in 'user_feedback.js' into the 'App.js' as:

App.js

```
import React from 'react';
import ReactDOM from 'react-dom/client';

// import all images
import avataruser1 from './images/batman.png';
import avataruser2 from './images/businessman.png';
import avataruser3 from './images/lady.png';

// import files
import User_feedback from './user_feedback';
import './index.css';
import User from './comments'

const App = function(){
  return(
   <>
    <h1 style={{textAlign:"center"}}>Student's full name</h1>
    <section className='container'>

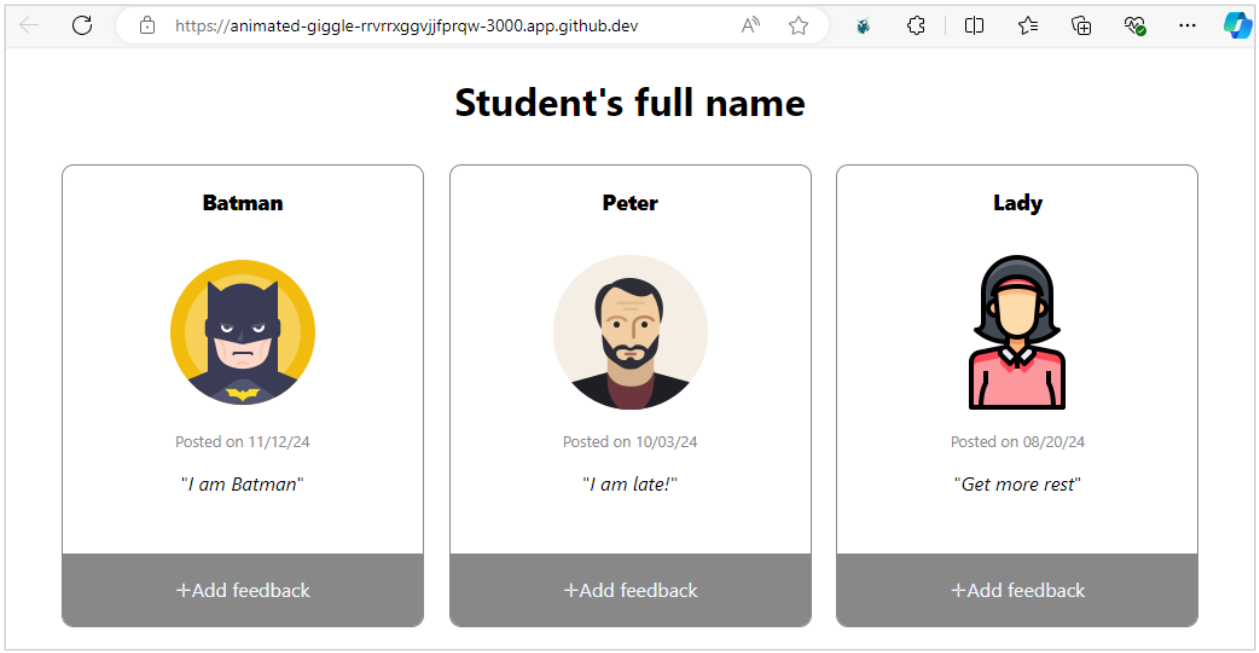
     {/* user 1 */}
     <User_feedback username="Batman"><User image={avataruser1}
     date="11/12/24" msg="I am Batman"/></User_feedback>

     {/* user 2 */}
     <User_feedback username="Peter"><User image={avataruser2}
     date="10/03/24" msg="I am late!"/></User_feedback>

     {/* user 3 */}
     <User_feedback username="Lady"><User image={avataruser3}
     date="08/20/24" msg="Get more rest"/></User_feedback>

   </section>
 </>
 )
}
```

The final app will appear in the browser as shown below:



5.6.2. Adding event listener

To add an event in React, you'll attach event handlers to your React components. Event handlers in React are defined as functions, and they can be added as props to elements.

Here's a step-by-step guide:

1. **Set Up Your React Component**

Create a React component where the event will be added.
import React from 'react';

```
function App() {
    return (
      <div>
        <button>Click Me</button>
      </div>
    );
  }

  export default App;
```

2. **Define an Event Handler**

Create a function in your component to handle the event.

```
function App() {
    const handleClick = () => {
      alert('Button clicked!');
    };

    return (
      <div>
        <button>Click Me</button>
      </div>
    );
  }
```

3. **Attach the Event**

Pass your event handler to the element as a prop. The event name should be camelCase (e.g., `onClick`).

```
function App() {
    const handleClick = () => {
      alert('Button clicked!');
    };

    return (
      <div>
        <button onClick={handleClick}>Click Me</button>
      </div>
    );
  }
```

4. **Use Inline Handlers (Optional)**

You can define the event handler inline, though this is generally not recommended for more complex handlers.

```
function App() {
    return (
```

```
      <div>
        <button onClick={() => alert('Button clicked!')}>Click
Me</button>
      </div>
    );
  }
```

5. **Pass Arguments to Event Handlers**

Use an arrow function or .bind() to pass arguments to the event handler.

```
function App() {
    const handleClick = (message) => {
      alert(message);
    };

    return (
      <div>
        <button onClick={()=>handleClick('Hello from
React!')}>Click Me
       </button>
      </div>
    );
  }
```

Supported Events in React

React supports many DOM events, such as:

- Mouse events: `onClick`, `onDoubleClick`, `onMouseEnter`, `onMouseLeave`, etc.
- Form events: `onChange`, `onSubmit`, `onFocus`, `onBlur`, etc.
- Keyboard events: `onKeyDown`, `onKeyPress`, `onKeyUp`.
- And more: `onScroll`, `onLoad`, `onError`.

Continue example 3)

For our app, we will add an event to the **"Add Feedback"** button so that when it is clicked, a modal window opens to submit feedback. The modal will display the name of each user, and all feedback will be shown in a list. Each feedback entry in the list should include the **user name**, the **feedback message**, and the **date** it was posted.

Modalwindow.js

```
import React from "react";
import './index.css'

const Modalwindow = function () {

  // function to close the modal window
  const closemodalwindow = () => {
     const modalwindow = document.querySelector(".modalwindow")
     modalwindow.style.display = "none";
  }

return (
   <>
     {/* Modal window */}
     <section className="modalwindow">
       <div className="modalcontent">
          <header className="modalheader">
            <p>Add Feedback
               <span className="modalusername"></span>
            </p>

            <p className="closemodal" onClick={closemodalwindow}>
               &#x58;</p>
          </header>

          <main className='modalbody'>
            <input className="commentarea" type="text"
               placeholder="Type your comments..."/>
            <button className="btnpostfeedback">Post Feedback
            </button>
          </main>
        </div>
      </section>
  </>
 )}
export default Modalwindow
```

App.js

```
import React from 'react';
import ReactDOM from 'react-dom/client';
```

```
// import all images
import avataruser1 from './images/batman.png';
import avataruser2 from './images/businessman.png';
import avataruser3 from './images/lady.png';

// import files
import User_feedback from './user_feedback';
import './index.css';
import User from './comments'

const App = function () {
  return (
    <>
      <h1 style={{textAlign:"center" }}>Student's full name</h1>

      <section className='container'>

        {/* user 1 */}
        <User_feedback username="Batman">
          <User image={avataruser1} date="11/12/24" msg="I am
          Batman"/>
        </User_feedback>

        {/* user 2 */}
        <User_feedback username="Peter">
          <User image={avataruser2} date="10/03/24" msg="I am
          late!" />
        </User_feedback>

        {/* user 3 */}
        <User_feedback username="Lady">
          <User image={avataruser3} date="08/20/24" msg="Get more
          rest" /></User_feedback>

     </section>

      {/* POST COMMENTS */}
      <section className='postcomment'>
          <ul className='commentlist'>
            <li>testing</li>
          </ul>
      </section>
```

```
    </>
  )
}
```

App.css

```
.modalwindow{
  background-color: rgba(200,200,200,0.7);
  position: fixed;
  top:0%; bottom: 0%; right: 0%; left: 0%;
}
.modalcontent{
  width: 500px; height: 300px; margin: 100px auto; background-
color: white
}
.modalheader{
  border-bottom: ridge 1px lightgray;
  display: flex;
  flex-wrap: wrap;
  padding: 0.1em 2em;
  color: #888;
  font-weight: bolder;
}
.modalheader p:nth-child(1){ font-size: 1.1em; }
.modalheader p:nth-child(2){ margin-left: auto;}
.commentarea{
  margin: 2em;   padding: 10px 15px;
  border-radius: 5px;
  border: solid 1px lightgray;
  width: 90%;
}
.btnpostfeedback{
  margin: 2em;
  background-color: #888;
  text-align: center;
  border: none;
  padding: 10px 20px;
  color:white;
  border-radius: 5px;
}
.btnpostfeedback:hover{
  background-color: crimson;
```

```
  cursor: pointer;}

/* POST COMMENTS */
.postcomment{
    width: 980px;
    background-color: rgb(240,240,240);
    margin: 3rem auto;
}
.commentlist li{
    list-style: none;
    background-color: white;
    padding: 1rem;
    border: solid 1px rgb(240,240,240);
}
```

After the modal window is styled, we can hide it by setting its display property to none.

```
.modalwindow{
  background-color: rgba(50,50,50,0.7);
  position: fixed;
  top:0%; bottom: 0%; right: 0%; left: 0%;
  display:none;
}
```

To open the modal window for each user, we can add an event handling to open the modal window in the user_feedback.js as:

user_feedback.js

```
import React from "react";
import './index.css';
import Modalwindow from "./modalwindow";

const User_feedback = function (props) {
    // open modal window
    const openmodalwindow = () => {
        const modalwindow= document.querySelector(".modalwindow")
        modalwindow.style.display = "block";
    }

    return (
      <>
```

```
        <div className="feedbackcontainer">
           <section className="feedbackcard">
             <div className="content">
                    <p>{props.username}</p>
             </div>
             <div className="description">{props.children}</div>
             <div className="cardfooter">
                <p className="addicon" onClick={openmodalwindow}>
                    <span>&#10011;</span>Add feedback</p>
             </div>
           </section>
        </div>

        {/* MODAL WINDOW */}

        <Modalwindow /> </>
    )
}
export default User_feedback
```

Next, we will display each user's name in their respective modal window when it is opened. There are different techniques in React to achieve this; however, for this app, we will use **props** to pass the user's name from the modal window to the user_feedback.js file.

user.feedback.js

```
import React from "react";
import './App.css'
import Modalwindow from './Modalwindow'

const User_feedback = function(props){
    // open modal window
    const openmodalwindow = () => {
        const modalwindow=document.querySelector(".modalWindow");
        modalwindow.style.display = "block";

     //set the user name to each modal window
     const modalusername =
     document.querySelector(".modalusername").innerHTML =
     `<i>${props.username}</i>`
    };

    return(
```

```
      <>
        <div className="feedbackcontainer">
          <section className="feedbackcard">
             <div className="content">{props.username}</div>
             <div className="description">{props.children}</div>
             <div className="cardfooter">
                    <p className="addicon" onClick={() =>
                         openmodalwindow()}>
                         <span>&#10011;</span> Add Feedback
                    </p>
                </div>
            </section>
        </div>

        {/* modal window */}
        <Modalwindow/>
        </>
    );};
export default User_feedback;
```

After that, we can add each feedback entry to the comment list, clear the input area, and close the modal window. To accomplish this, we will create a function in the modalwindow.js file that runs when the **Add Feedback** button is pressed. This function will collect the text entered in the modal's input field and add it to the comment list. Each posted entry will display the **user's name**, the **comment**, and the **date** it was submitted.

modalwindow.js

```
import React from "react"
import "./App.css"

const Modalwindow = function (props) {
    // function to close the modal window
    const closemodalwindow = ()=>{
       const modalwindow = document.querySelector(".modalWindow")
       modalwindow.style.display = "none"
    }

    // function to collect comments
    const collectcomment = ()=>{
        let commentarea = document.querySelector(".commentarea")
        const commentlist =document.querySelector(".commentlist")
```

```
        const modalusername =
          document.querySelector(".modalusername").textContent
        const postedate = new Date().toLocaleDateString()
        const modalwindow =document.querySelector(".modalWindow")

        /* append comment to the list */
        commentlist.innerHTML += `<li><i><b>${modalusername}—
          </b></i> ${commentarea.value}
          <span style="font-size:0.8rem; color: gray;
          font-style: italic">posted ${postedate}</span> </li>`

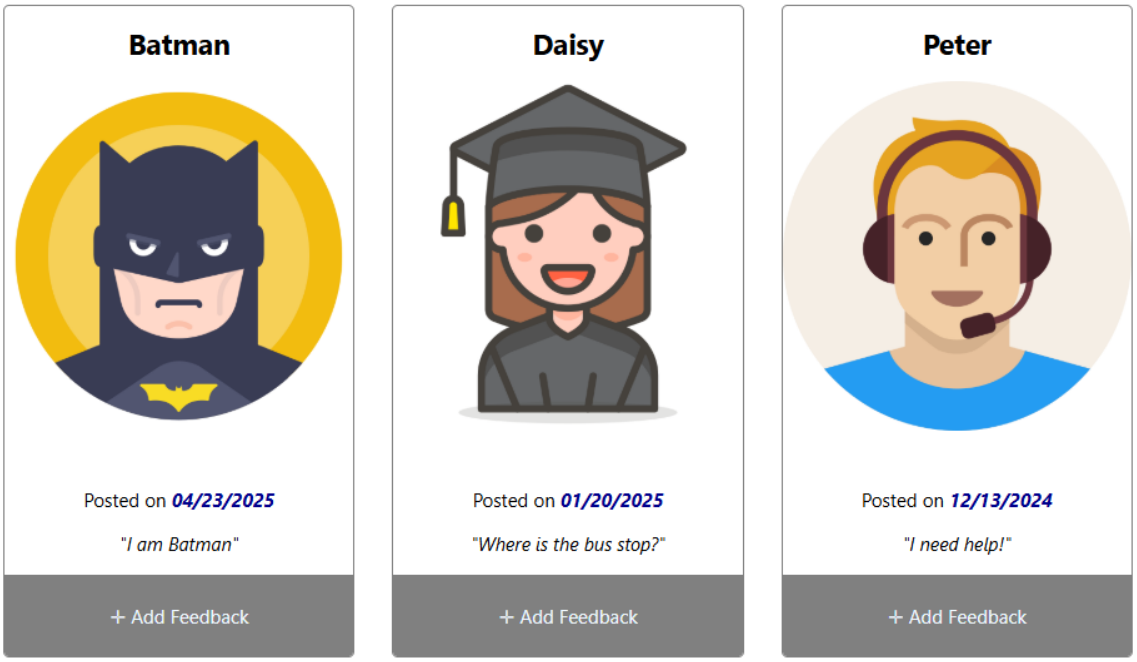
        /* after the feedback is posted, close the modal window
        and clear the comment area */

        commentarea.value = ""
        modalwindow.style.display = "none"
    }

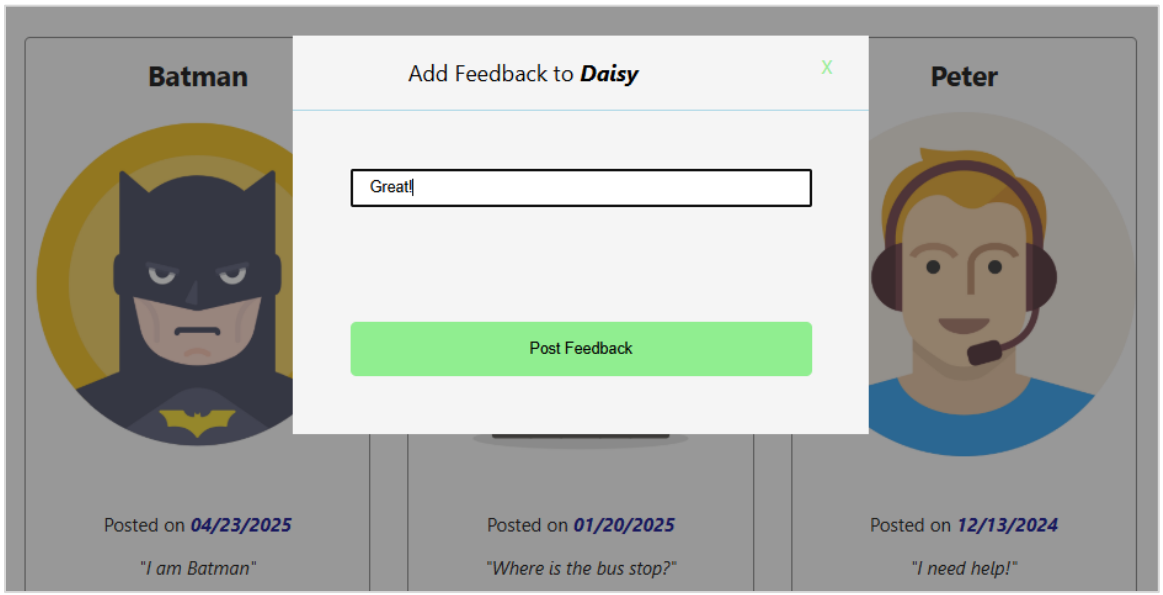
    return (
      <>
         {/* modal window */}
         <section className="modalWindow">
         <div className="modalcontent">
            <header className="modalheader">
               <p>Add Feedback <span className="modalusername">
               </span></p>
            <p classNam="closemodal" onClick={closemodalwindow}>
               &#x58;</p>
          </header>

          <main className="modalbody">
             <input className="commentarea" type="text"
               placeholder="Type your comments..." />
             <button className="btnpostfeedback"
               onClick={collectcomment} >Post Feedback</button>
          </main>
         </div>
        </section>
      </>
    )
}
export default Modalwindow
```

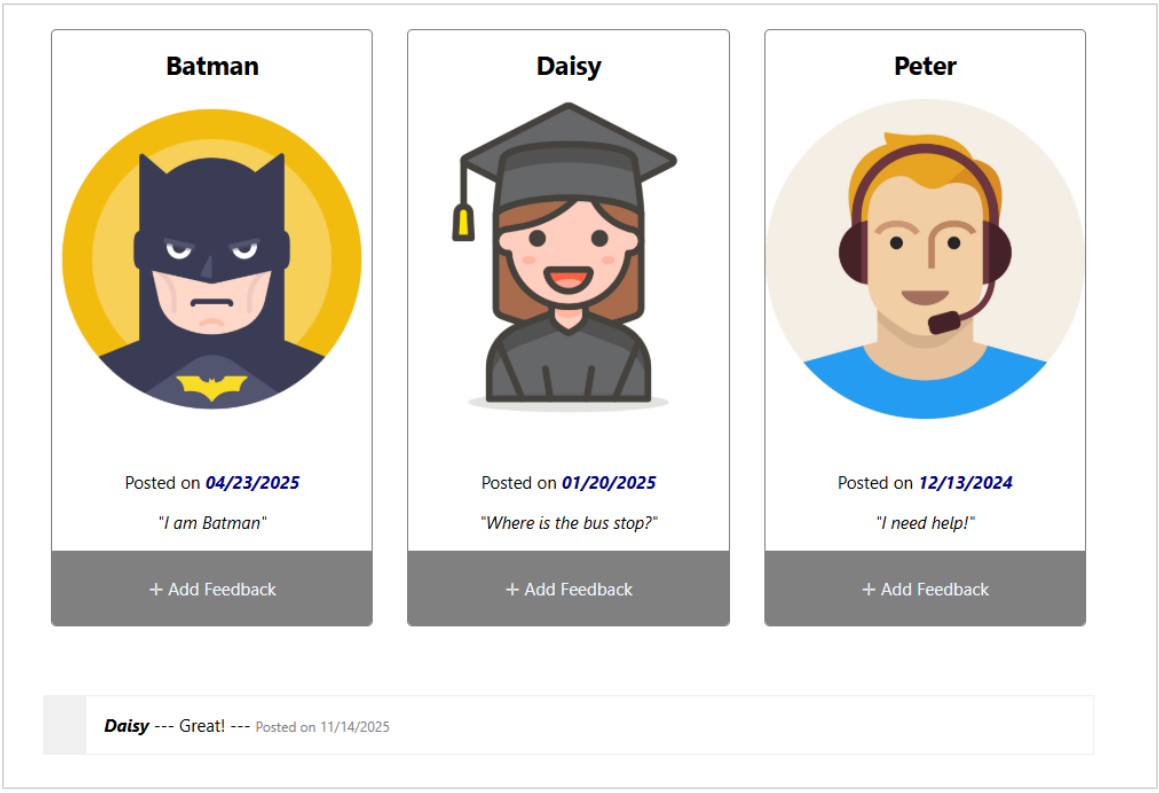
When the application loads, three cards are displayed on the screen. If the user wants to add a comment for Daisy, they can click the **"Add Feedback"** button on Daisy's card. This action will open a modal window where the user can submit their comment.



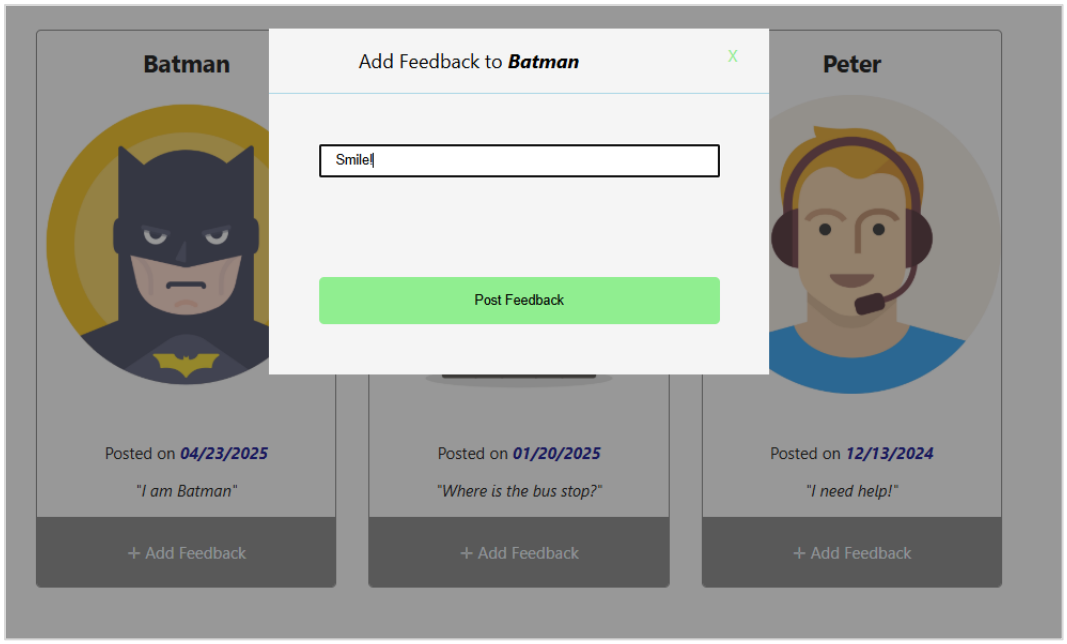
In the modal window, when the user enters "Great!" as a comment and clicks the **"Post Feedback"** button:

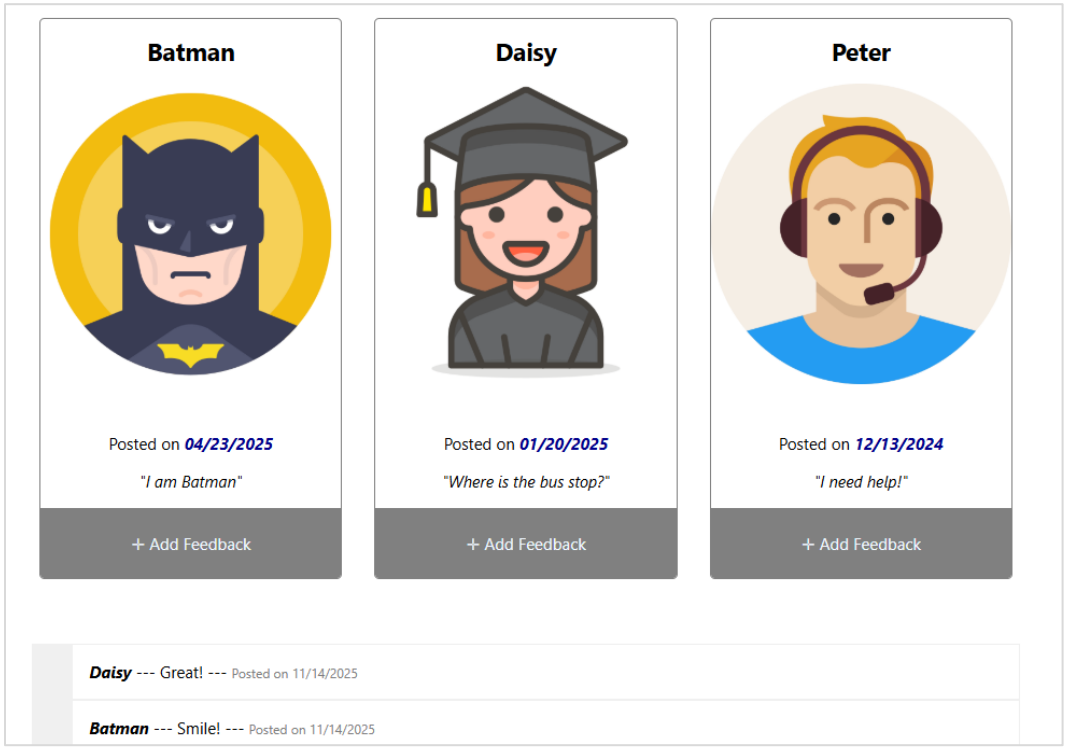


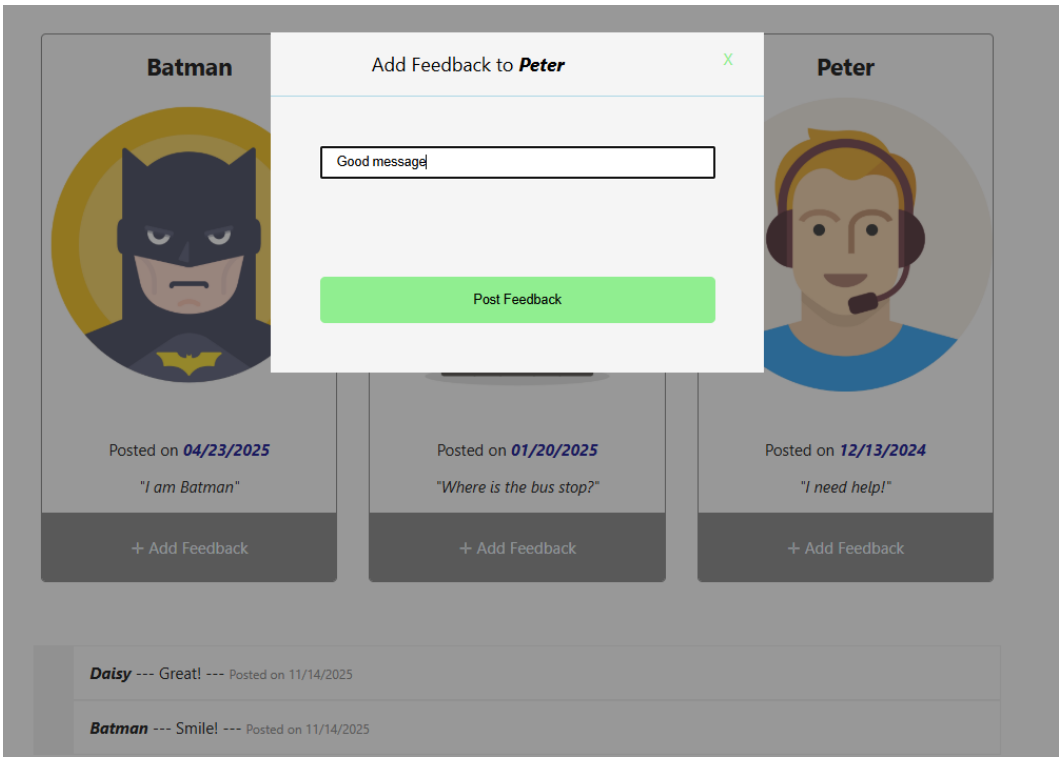
The comment will appear in the comment list below the three cards, with the name **Daisy** displayed next to it.

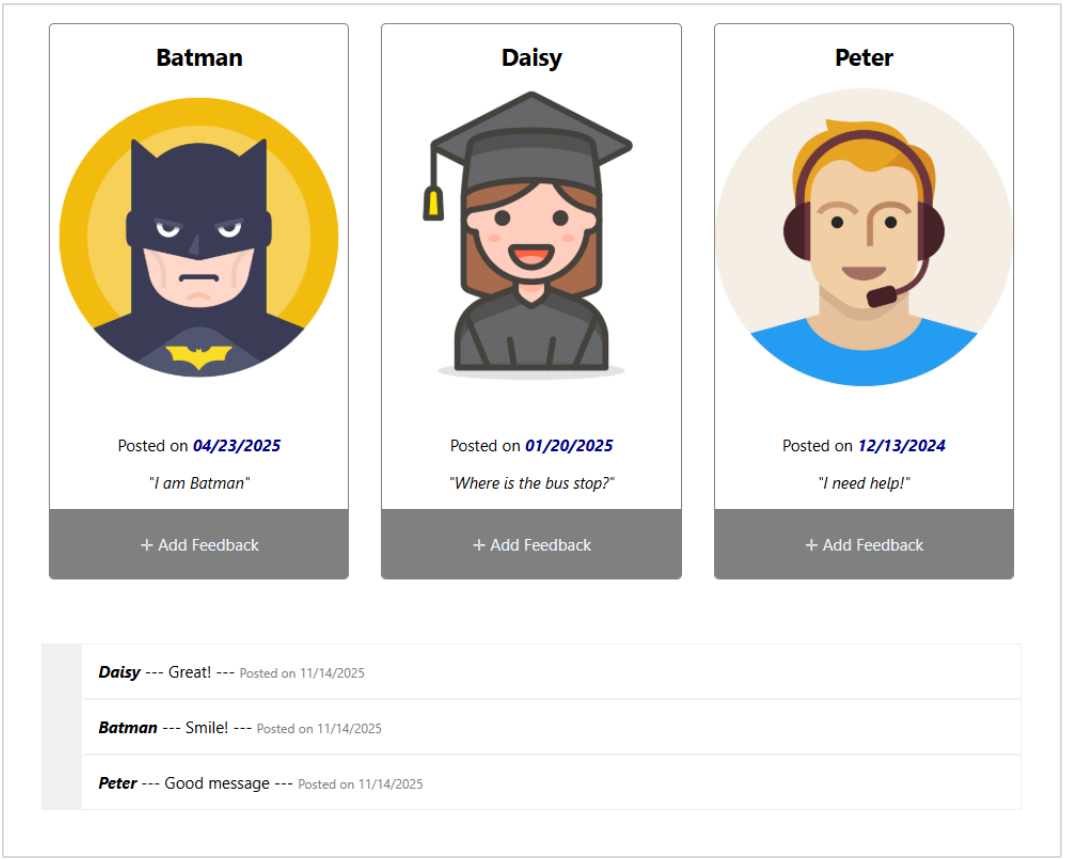


Similarly, if you add a comment to Peter or Batman, their comments will also be displayed in the comment list, as illustrated in the four images below. The first two images show the feedback added to Peter, while the next two images display the feedback submitted for Batman.


Batman
Add Feedback to Batman
X
Peter
Smile!
Post Feedback
Posted on 04/23/2025
"I am Batman"
+ Add Feedback
Posted on 01/20/2025
"Where is the bus stop?"
+ Add Feedback
Posted on 12/13/2024
"I need help!"
+ Add Feedback


Batman
Daisy
Peter
Posted on 04/23/2025
"I am Batman"
+ Add Feedback
Posted on 01/20/2025
"Where is the bus stop?"
+ Add Feedback
Posted on 12/13/2024
"I need help!"
+ Add Feedback
Daisy --- Great! --- Posted on 11/14/2025
Batman --- Smile! --- Posted on 11/14/2025


Batman
Add Feedback to Peter
X
Peter
Good message
Post Feedback
Posted on 04/23/2025
"I am Batman"
+ Add Feedback
Posted on 01/20/2025
"Where is the bus stop?"
+ Add Feedback
Posted on 12/13/2024
"I need help!"
+ Add Feedback
Daisy --- Great! --- Posted on 11/14/2025
Batman --- Smile! --- Posted on 11/14/2025


Batman
Daisy
Peter
Posted on 04/23/2025
"I am Batman"
+ Add Feedback
Posted on 01/20/2025
"Where is the bus stop?"
+ Add Feedback
Posted on 12/13/2024
"I need help!"
+ Add Feedback
Daisy --- Great! --- Posted on 11/14/2025
Batman --- Smile! --- Posted on 11/14/2025
Peter --- Good message --- Posted on 11/14/2025

5.7. Application of React States

In React, state is an object that determines how a component renders and behaves. State is managed within the component (it is local or encapsulated) and can change over time, typically in response to user actions or network responses.

5.7.1. React States Components

Functional Components with Hooks

In functional components, state is managed using the `useState` hook.

1. **Initialization and State Setter:**

```
import React, {useState} from "react"

function App() {
  // set the initial state
  const[count, setCount] = useState(0)

  return (
    <>
      <h1>Lab 14, React state</h1>
      <p>Count = {count}</p>
      <button onClick={()=>setCount(count+1)}>Increment the
          count</button>
      <button onClick={()=>setCount(0)}>Reset button</button>
    </>
  );
}

export default App;
```

Class Components

In class components, state is initialized in the constructor and updated using the **setState** method.

1. **Initialization:**

```
class MyComponent extends React.Component {
  constructor(props) {
    super(props);
    this.state = {
      count: 0
    };
  }}
```

2. **Updating State:**

```
class MyComponent extends React.Component {
  constructor(props) {
    super(props);
    this.state = {
      count: 0
    };
  }

  increment = () => {
    this.setState({ count: this.state.count + 1 });
  }

  render() {
    return (
      <div>
        <p>Count: {this.state.count}</p>
        <button onClick={this.increment}>Increment</button>
      </div>
    );
  }
}
```

Key Points

- **Encapsulation:** State is private to the component where it is declared.
- **Initialization:** State is initialized when a component is created.
- **Updating:** State should be updated using setState (class components) or the state setter function returned by useState (functional components).
- **Asynchronous:** State updates may be asynchronous and batched for performance optimization.
- **Triggers Re-renders:** When state changes, the component re-renders to reflect the new state.

Example with Multiple States

You can manage multiple pieces of state by calling useState multiple times.

```
function MyComponent() {
  const [count, setCount] = useState(0);
  const [name, setName] = useState('John');

  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={() => setCount(count + 1)}>Increment
          </button>
      <p>Name: {name}</p>
      <input type="text" value={name} onChange={(e) =>
          setName(e.target.value)} />
    </div>
  );
}
```

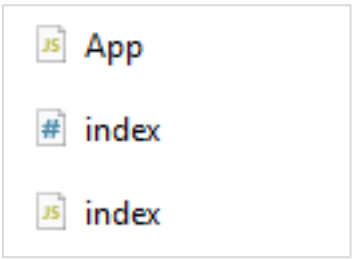
React state is a powerful tool for managing dynamic data in your application and responding to user interactions in a declarative way.

Example 4) React state app: slice cards

For this project, we are going to create slice cards using React states. We can create a project named '**sliceapp**'

```
npx create-react-app sliceapp
```

After we create the react app project folder, we can clean up the files and codes that are automatically generated. For this app, we are going to be using 'index.js', 'App.js', and 'index.css' files, therefore, we can clean up our 'src' folder to keep only the files that we need:



Now, we can clean each of the files as:

index.js

```
import React from 'react';
import ReactDOM from 'react-dom/client';
import './index.css';
import App from './App'; // Import the App component

const root =
ReactDOM.createRoot(document.getElementById('root'));
root.render(<App />);
```

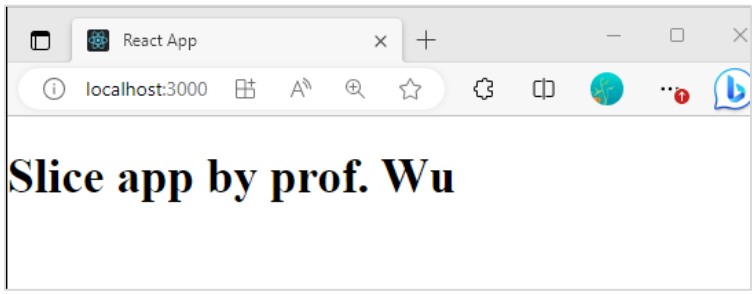
App.js

```
function App() {
  return (
    <>
    <h1>Slice app by prof. Wu</h1>
    </>
  );
}
export default App;
```

index.css

```
body{
  margin: 0%;
  padding: 0%;
  box-sizing: border-box;
}
```

The browser should show:



Now, we can start creating the structure or layout of the app in the 'App.js' file:

App.js

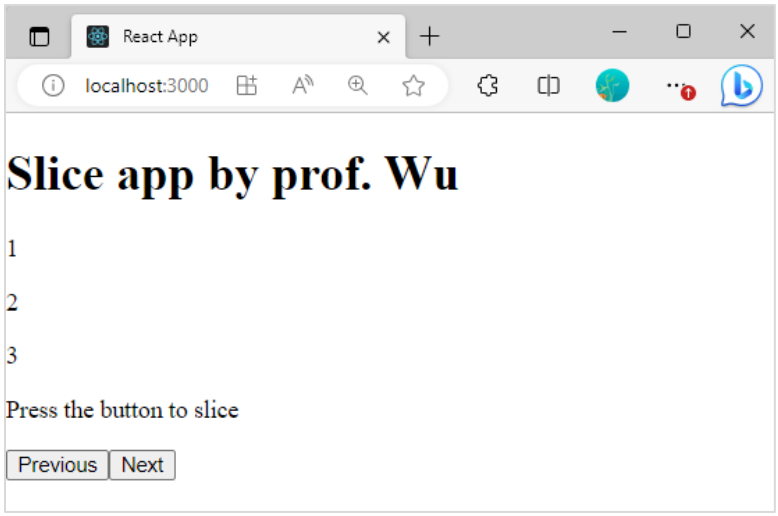
```
function App() {
  return (
    <>
    <h1 className="title">Slice app by prof. Wu</h1>

    <section className="slicewrapper">

      <div className="numberWrapper">
        <div><p>1</p></div>
        <div><p>2</p></div>
        <div><p>3</p></div>
      </div>
      <p className='msg'>Press the button to slice</p>

      <div className="btngroup">
        <button>Previous</button>
        <button>Next</button>
      </div>
    </section>
    </>
  );
}
export default App;
```

The browser should show:



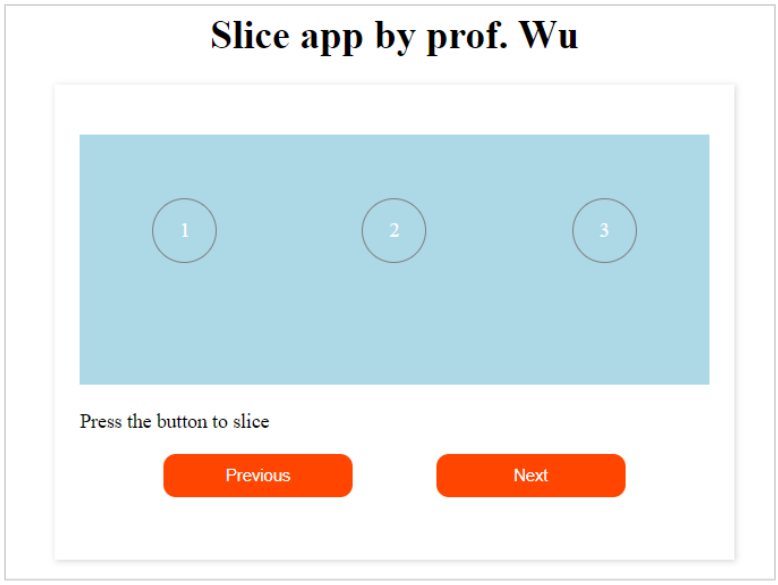
Styling the app in the 'App.css' file:

App.css

```
body{
  margin: 0%;
  padding: 0%;
  box-sizing: border-box;
}
.title{
  text-align: center;
}
/* ------- SLICE APP ------- */
.slicewrapper{
  width: 600px;
  margin: 2rem auto;
  box-shadow: 1px 1px 5px lightgray;
  padding: 20px;
}
.numberWrapper{
  height: 200px;
  display: flex;
  justify-content: space-around;
  margin: 2rem auto;
  background-color: lightblue;
  color: white;}
.numberWrapper>div{
  border: solid gray 1px;
```

```
  width: 50px;
  height: 50px;
  border-radius: 50%;
  text-align: center;
  margin: 3rem;
}
.msg{
  text-align: center;
  font-weight: bolder;
}
.btngroup{
display: flex;
justify-content: space-evenly;
margin-bottom: 30px;
}
.btngroup > button{
  width: 30%;
  background-color: orangered;
  color:white;
  border: none;
  padding: 1rem;
  border-radius: 10px;
}
.btngroup > button:hover{
  background-color: darkblue;
  box-shadow: 1px 1px 5px gray;
  font-weight: bolder;
}
.active{
  background-color: orangered;
  box-shadow: 0px 0px 5px 1px gray;
}
```

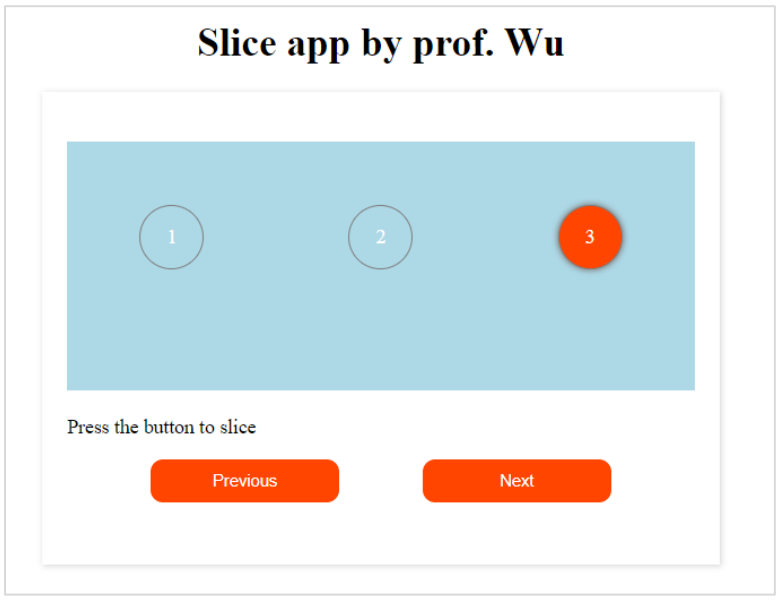
The internet browser will look as:



We can test the 'active' class by adding a class name to the third

```
<div className="active"><p>3</p></div>
```

The browser to show:



After it, we are going to create an array list of three messages that will print the message in the paragraph that now has the message 'Press the button to slice'. We also need to set the initial counter to print each message from 1:

App.js

```
// list of three messages
const messages = ["I think, therefore I am", "I have a dream",
"The only thing we have to fear is fear itself"]

function App() {
  //set up an initial counter for the slice
  let counter = 1;

  return (
    <>

      <h1 className="title">Slice app by prof. Wu</h1>
      <section className="slicewrapper">
        <div className="numberWrapper">
          <div ><p>1</p></div>
          <div><p>2</p></div>
          <div className="active"><p>3</p></div>
        </div>

        <p> Press the button to slice</p>
        <div className="btngroup">
          <button>Previous</button>
          <button>Next</button>
        </div>
      </section>
    </>
  );
}

export default App;
```

After it, we display one message at the time in the paragraph using curly braces {}

```
// list of three messages
```

```
const messages = ["I think, therefore I am", "I have a dream",
"The only thing we have to fear is fear itself"]

function App() {
  //set initial counter
  let counter = 1;

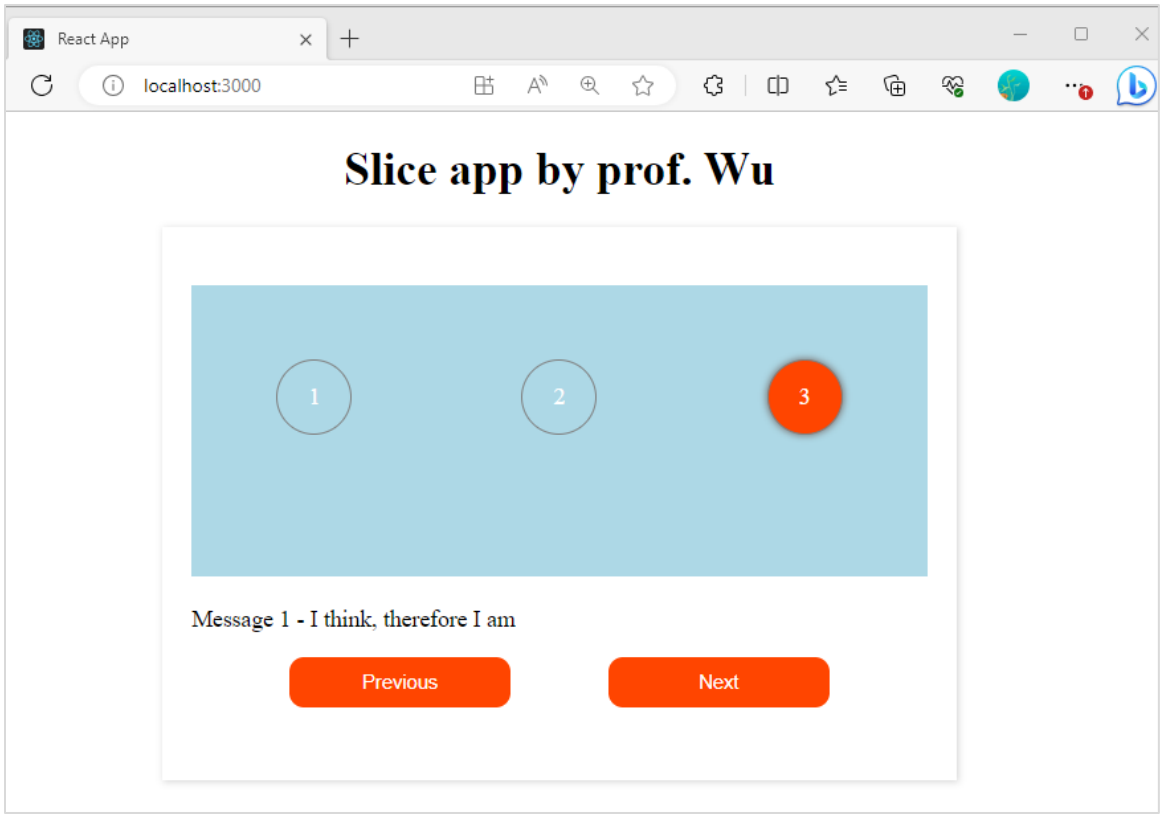
  return (
    <>

      <h1 className="title">Slice app by prof. Wu</h1>
      <section className="slicewrapper">
        <div className="numberWrapper">
          <div ><p>1</p></div>
          <div><p>2</p></div>
          <div className="active"><p>3</p></div>
        </div>

        <p>Message {counter} - {messages[counter - 1]}</p>
        <div className="btngroup">
          <button>Previous</button>
          <button>Next</button>
        </div>
      </section>
    </>
  );
}

export default App;
```

The browser should show:



5.7.2. Event handler in React

After we have our design ready, the next thing we are going to do is to apply events to our app.

To test the functionality of the event listener in React, we simply collect the element that we want to trigger and then add an event to it. For example, we want to add an event that when the 'previous' button is pressed, it will open a dialog box saying 'Previous button was clicked':

```
// list of three messages
const messages = ["I think, therefore I am", "I have a dream",
"The only thing we have to fear is fear itself"]

function App() {
  //set initial counter
  let counter = 1;


return (
    <>
```

```
      <h1 className="title">Slice app by prof. Wu</h1>
      <section className="slicewrapper">
        <div className="numberWrapper">
          <div ><p>1</p></div>
          <div><p>2</p></div>
          <div className="active"><p>3</p></div>
        </div>

        <p>Message {counter} - {messages[counter - 1]}</p>
        <div className="btngroup">
          <button onClick={function(){alert("PREVIOUS
button")}}>Previous</button>

          <button onClick={function(){alert("NEXT
button")}}>Next</button></div>
      </section>
    </>
  );
}

export default App;
```

5.7.3. Use of state

After our App is set up, the next step is to set up the state to our app. State is a JavaScript object that holds data inside a component. We can think of state as a mechanism that allows a component to be dynamic and interactive since it automatically enables the data that comes to the screen when the component is first rendered to change based on user experiences and interactions.

React components often contain data that changes with user interactions or external data changes. The concept of state is used to track these data changes.

State is created using the **useState** hook.

State makes react components dynamic and interactive. It basically ensures that the interface is updated immediately after user interactions or any data changes.

State stores data associated with a specific component, which can be then processed and used within the same component, or even sent to other components as props.

State can be passed from one component to another. It optimizes interface updates using react virtual Dom.

Using state only when needed to update the interface will enhance performance.

Using this data also makes the code way more organized, simple and structured.

Continuation of example 4) In our app, we will use state to update the message whenever the **Next** or **Previous** button is clicked.

First in our App.js file, we need to import the useState object to our app.

```
import {useState} from 'react'
```

After it, we are going to set the initial state to '1'

App.js

```
import {useState} from 'react'
// list of three messages
const messages = ["I think, therefore I am", "I have a dream",
"The only thing we have to fear is fear itself"]



function App() {
  //set initial counter
  let counter = 1
  // set initial state
  const arr = useState(1)
  // console state
  console.log(arr)

  return (
    <>

      <h1 className="title">Slice app by prof. Wu</h1>

      <section className="slicewrapper">
        <div className="numberWrapper">
          <div ><p>1</p></div>
```

```
          <div><p>2</p></div>
          <div className="active"><p>3</p></div>
        </div>

        <p>Message {counter} - {messages[counter - 1]}</p>
        <div className="btngroup">
          <button className="previousbtn">Previous</button>
          <button className="nextbtn">Next</button>
        </div>
      </section>

    </>
  );
}

export default App;
```

The **useState** hook returns an array with two elements: the first is the current state value (for example, initialized to 1), and the second is the function used to update that state.

If you change the initial value to 5, like this: **`const arr = useState(5);`**

you will see in the console that the first element of the state array is now 5.

With this concept in mind, let's move on to setting up the state with an initial value called **step** and the state update function named **setStep**.

```
const [counter, setCounter] = useState(1)
```

After that, we add the **setCounter** state to the event handlers for both the **Next** and **Previous** buttons.

```
import {useState} from 'react'
// list of three messages
const messages = ["I think, therefore I am", "I have a dream",
"The only thing we have to fear is fear itself"]

function App() {
```

```
   // set initial state
   const [counter,setCounter] = useState(1)

  return (
    <>

      <h1 className="title">Slice app by prof. Wu</h1>

      <section className="slicewrapper">
        <div className="numberWrapper">
          <div><p>1</p></div>
          <div><p>2</p></div>
          <div><p>3</p></div>
        </div>

        <p>Message {counter} - {messages[counter - 1]}</p>
        <div className="btngroup">
          <button onClick={function()
                {if(counter>1)setCounter(counter-1)}}>Previous
          </button>
       <button onClick={function()
          {if(counter<messages.length)setCounter(counter+1)}>
          Next
        </button>


        </div>
     </section>

    </>
  );
}

export default App;
```

The if statement in the **Next** button prevents the app from going beyond the third message, while the if statement in the **Previous** button ensures the app does not go before the first message.

As a final step, we use a logical expression to apply the active class to the message that is currently being displayed. For example, when the second message is shown, the containing the number 2 will have the class active:

```
<div className={counter==1 ? "active":"" }>1</div>
<div className={counter==2 ? "active":"" }>2</div>
<div className={counter==3 ? "active":"" }>3</div>
```

Open and close button app

Use React state to create a button that toggles a section open and closed.

```
import {useState} from 'react'

function App() {

  // OPEN AND CLOSE BUTTON
  const [isopen, setIsopen] =useState(true)
  return (
    <>

      <h1 className='title'>Open and close button</h1>
      <div className='info'>
          <p><b>Cat (Felis catus)</b>, commonly referred to as
          the domestic cat or house cat</p>
     </div>
      <section>
        {
          isopen &&
          (
            <div className='readmore'>
               <p>The cat (Felis catus), commonly referred to as
               the domestic cat or house cat, is a small
               domesticated carnivorous mammal. It is the only
               domesticated species of the family Felidae. Recent
               advances in archaeology and genetics have shown
               that the domestication of the cat occurred in the
               Near East around 7500 BC</p>
            </div>
          )
        }
      </section>
```

```
    </>
  );
}

export default App;
```

If we change the initial state to false:

```
const [isopen, setIsopen] =useState(false)
```

The will be hidden by default. We will then use React state to manage its visibility by setting the state to either true or false. To achieve this, we will create a button that toggles the state between true and false each time it is clicked.

```
import {useState} from 'react'

function App() {
  // OPEN AND CLOSE BUTTON
  const [isopen, setIsopen] =useState(false)

  // function to open a read more paragraph
  document.querySelector(".readmorebtn").addEventListener("click",
function(){
    setIsopen(!isopen)
  })
  return (
    <>
      <h1 className='title'>Open and close button</h1>
      <div className='info'>
           <p><b>Cat (Felis catus)</b>, commonly referred to as the
           domestic cat or house cat <button className='readmorebtn'>
           Read More </button>
            </p>
      </div>
      <section>
        {
          isopen &&
          (
            <div className='readmore'>
                <p>The cat (Felis catus), commonly referred to as the
                domestic cat or house cat, is a small domesticated
                carnivorous mammal. It is the only domesticated
                species of the family Felidae. Recent advances in
```

```
                archaeology and genetics have shown that the
                domestication of the cat occurred in the Near East
                around 7500 BC
                </p>
            </div>
          )
        }
      </section>
    </>
  );
}

export default App;
```

Instead of using event listener, we can also use the inline event handler as:

```
<button onClick={function(){setIsopen(!isopen)}}>
```

Now, let's make the button more dynamic so it toggles between the labels **"Read More"** and **"Hide"**. The button should display **"Read More"** when the paragraph is hidden, and switch to **"Hide"** when the paragraph is visible.

```
<button className='readmorebtn'> {isopen ? "Hide" : "Read More"}
</button>
```

5.8. Shopping cart using react State

This session teaches learners how to build a simple shopping cart using React State. They will create a product list, add items to a cart, update quantities, and remove items, all while understanding how state changes dynamically update the user interface.

We'll build:

- A list of items to add to cart
- A Cart that shows added items
- Ability to **add**, **remove**, and **update quantity**

Learning Outcomes

Learners will be able to:

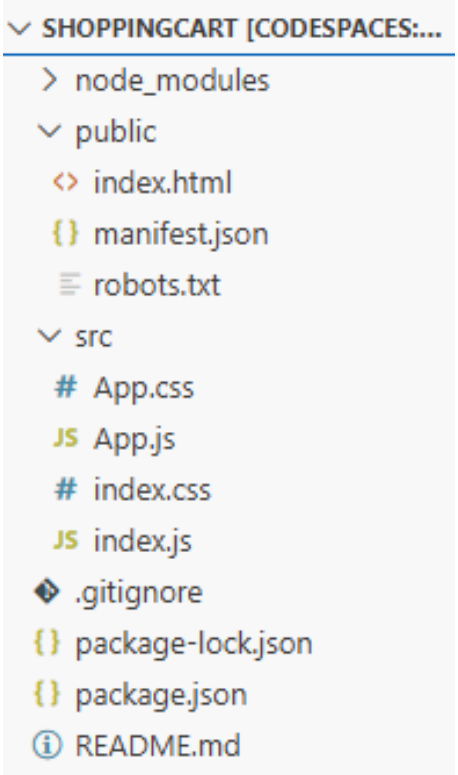
1. Use React State to manage dynamic data.
2. Display a list of products and add them to a cart.
3. Update cart item quantities using state.
4. Remove items from the cart.
5. Build a simple functional shopping cart that responds to user actions.

Part 1: Setting up the React environment and Starter Structure

- create a React app named 'shoppingcart'

```
npx create-react-app shoppingcart
```

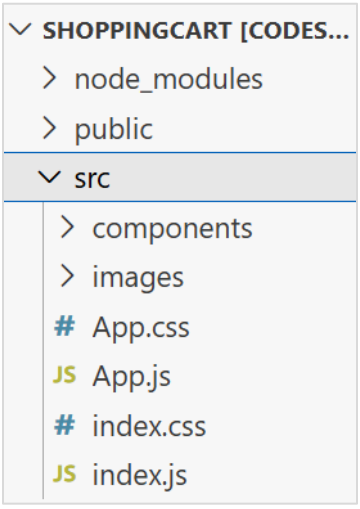
After it, change the directory in the Terminal to `shoppingcart`. The structure of the starter should be as:



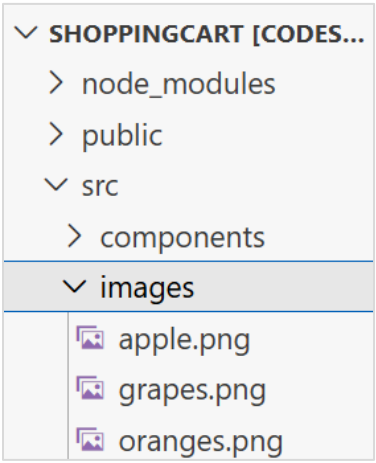
Now, within the **src** folder, we are going to create a new folder named **components.** Within the components folder, we are creating two files, **ProductList.js** and **Cart.js**.

```
src/
├── App.js
├── components/
│   ├── ProductList.js
│   └── Cart.js
└── index.js
```

We can also add a new folder named 'images' under the root folder.



Now we can download three images and save them in the **images** folder



Part 2. Create the product list

To create a product list, instead of using props, we can also use a list of items and use a map method to loop through each item in a list.

The **.map()** method in this lab is being used to **loop through the products array** and **render a block of JSX for each product**.

What .map() does here?

```
products.map((product) => { ... })
```

- **products** is an array (each item is a product object).
- **.map()** loops through that array.
- For **each** product, it:
    - Returns a **JSX card**
    - Filled with that product's data (image, name, prices, button)

React will then show **one card per product** on the screen.

Why we need key={product.id}

React requires a **unique key** for each item in a list to track which elements change, so rendering stays efficient.

ProductList.js

```
import React from "react";
import '../App.css';
import apple from '../images/apple.png'
import grapes from '../images/grape.png'
import orange from '../images/orange.png'

const products = [
  { id: 1, name: "Apple", price: 1 , was: 2.5, image:apple},
  { id: 2, name: "Grapes", price: 1.5, was: 2.8, image: grapes },
  { id: 3, name: "Orange", price: 3, was: 5.0, image: orange },
];
```

```
const ProductList = ({ addToCart }) => {
  return (
    <div>
      <h2 className="producttitle">Products</h2>
      <section className="cardcontainer">
        {products.map((product) => (
            <div key={product.id} className="card">
               <img src={product.image}/>
               <p className="productname">{product.name}
                 <s>{product.was}/lb</s> <span>
                 ${product.price}</span>/lb
               </p>
               <button className="btn_addcart" onClick={() =>
                    addToCart(product)}> Add to Cart
               </button>
            </div>
        ))}
      </section>
    </div>
  );
};

export default ProductList;
```

App.js

```
import './App.css';
import React, { useState } from "react";
import ProductList from "./components/ProductList";

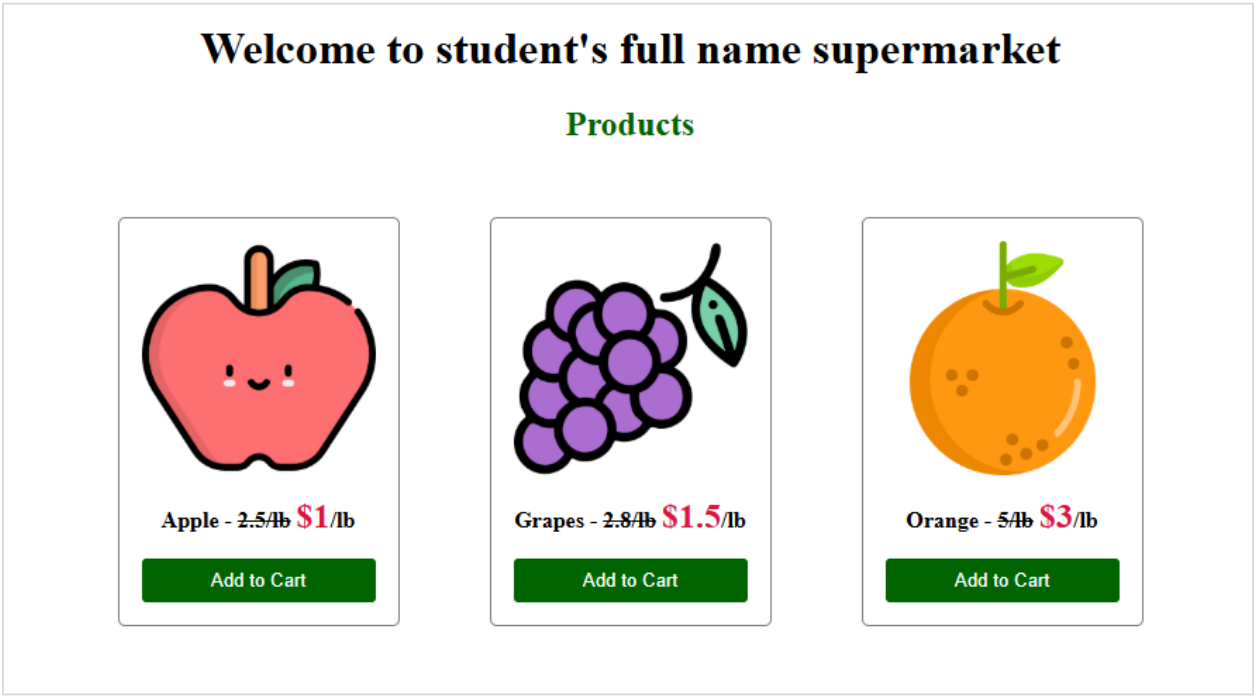
function App() {

  return (
    <div>
      <h1 className='apptitle'>Welcome to student's full name
      supermarket</h1>
      <ProductList addToCart={addToCart} />
    </div>
  );
}
export default App;
```

App.css

```
*{box-sizing: border-box;}
img{width: 100%; display: block;}
.apptitle{
  text-align: center;
}
/* productlist.js*/
.producttitle{text-align: center; color:darkgreen;}
.cardcontainer{
  display: flex;
  justify-content: center;
  flex-wrap: wrap;
}
.card{
  border: solid 1px gray;
  width: 200px;
  margin: 2rem;
  padding: 1rem;
  border-radius: 5px;
  transition: ease-in-out 500ms;
}
.productname{
  text-align: center;
  font-weight: bold;
}
.productname span{
  font-size: 1.5rem;
  color: crimson;
}
.btn_addcart{
  width:100%;
  border: none;
  background-color: darkgreen;
  color: white;
  padding: 0.5rem ;
  border-radius: 3px;
}
.btn_addcart:hover{
  box-shadow: 0px 0px 2px 1px gray;
  cursor: pointer;
  background-color: crimson;}
```

The browser should show the following front-end:



After it, we are going to implement:

- Add items
- Auto-increase quantity
- Remove items
- Display total price

To do so, we are going to create a function, **addToCart(),** where we are going to add items, auto-increase quantity. The flow chart of the function should be as the following diagram:

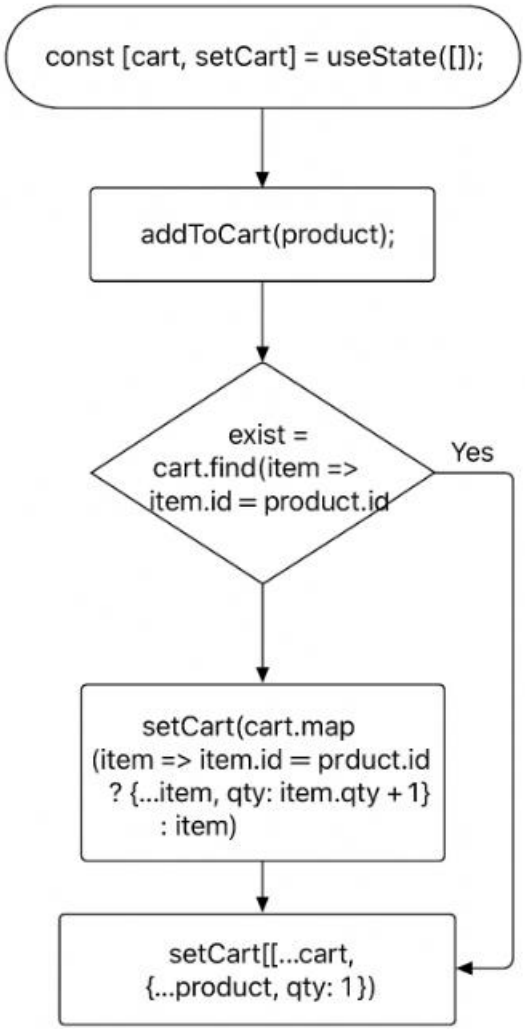


Diagram 1: flow chart of the function addToCart, to add and auto-increment quantity

App.js

```
import './App.css';
import React, { useState } from "react";
import ProductList from "./components/ProductList";
import Cart from "./components/Cart";

function App() {
  const [cart, setCart] = useState([]);

  {/*Find if the product already exists in the cart.*/}
  const addToCart = (product) => {
    const exist = cart.find((item) => item.id === product.id);
```

```
   {/* If product exists, increase quantity */}

      if (exist) {
        setCart(
          cart.map((item) => {
            if (item.id === product.id) {
              return { ...item, qty: item.qty + 1 };
            } else {
              return item;
            }
          })
        );
      }

     {/* If product does NOT exist, add new item */}
     else {
        setCart([...cart, { ...product, qty: 1 }]);
     }
    };


    return (
      <div>
        <h1 className='apptitle'>Welcome to student's full name
          supermarket</h1>
        <ProductList addToCart={addToCart} />

      </div>
    );
  }

  export default App;
```

Cart.js

```
import React from "react";
import '../App.css';

const Cart = ({ cart, removeFromCart }) => {
  return (
    <div>
      <h2 className="carttitle">Your Cart</h2>
```

```
      <section className="cartcontainer">
        {cart.length === 0 && <p className="emptymsg">
          Cart is empty</p>}
        {cart.map((item) => (
          <div key={item.id} className="itemcart">
            <img className="cartitemimg" src={item.image} />
            <p className="cartitemname">
              {item.name}
            </p>
            <p className="cartitemprice">
               ${item.price}/each
            </p>
            <p className="cartitemquantity">
              Quantity = {item.qty}
            </p>
            <p className="cartitemtotal">
              Total = {item.price*item.qty}
            </p>
           <button className="btncartremove" onClick={() =>
           removeFromCart(item.id)}>
              x Remove
            </button>
          </div>
        ))}

        <p className="totalcart">
          Total: $
         {cart.reduce((total, item) =>
               total+item.price*item.qty, 0)}
               {/*0 is the initial value for the total in the
               reduce method*/}
        </p>
      </section>

    </div>
  );
};

export default Cart;
```

style.css

```
/* Cart.js */
.carttitle{text-align: center; color:darkgreen;}
.cartcontainer{
  width: 1000px;
  margin: 2rem auto;
  border: solid 1px gray;
  padding: 1.5rem;
}
.itemcart{
  display: flex;
  margin-bottom: 1.5rem;
  font-size: 1.3rem;
}
.cartitemimg{
  width: 10%;
  padding: 0.5rem;
}
.cartitemname{
  padding-left: 2rem;
}
.cartitemprice, .cartitemquantity,.cartitemtotal, .cartitemname{
  width: 20%;
  padding:0.5rem;
}
.btncartremove{
 color:crimson;
 background-color: rgba(0,0,0,0);
 border:none;
 cursor: pointer;
 height: auto;
}
.btncartremove:hover{
  font-weight: bolder;
  text-decoration: underline;
}
.emptymsg{
  font-size: 1.5rem;
  font-weight: bolder;
  color: gray;
}
.totalcart{
```

```
  border-top: solid gray 1px;
  font-size: 1.5rem;
  font-weight: bolder;
  padding: 1rem 0rem;
  text-align: right;
}
```

App.js

```
const removeFromCart = (id) => {
      setCart(cart.filter((item) => item.id !== id));
    };

    return (
      <div>
        <h1 className='apptitle'>
          Welcome to student's full name supermarket
        </h1>
        <ProductList addToCart={addToCart} />
        <Cart cart={cart} removeFromCart={removeFromCart} />
```

What the following function does?

```
const removeFromCart = (id) => {
  setCart(cart.filter((item) => item.id !== id));
};
```

This function removes an item from the shopping cart based on its id.

- The function receives an id as: `(id) => {  }`
  This is the ID of the product the user wants to remove.
- `setCart` is the React state update function for your cart state. This updates the cart with a new array.
- `cart.filter((item) => item.id !== id)` filters the cart:
  - `.filter()` loops through every item in the cart
  - It keeps only the items whose id does NOT match the one passed in
  - The item whose id does match gets removed

In other words, create a new cart array that contains everything except the item with this ID.

Exercise 4: Practicing React State

Objective: Learn how to use **React state (useState)** to create interactive components that update dynamically based on user actions.

Project: Simple Counter & Toggle App

You will build a small React application with **three mini-features** using state:

1. Counter (Increase / Decrease)
2. Show/Hide Text
3. Dynamic Message Switcher

Part 1: Counter App

Task: Create a counter that increases and decreases when buttons are clicked.

Requirements:

- Start value should be 0
- Button: **Increase**
- Button: **Decrease**
- Display current value

AI Prompt Help:

Ask AI: "Show me how to build a counter using useState in React with increment and decrement buttons."

Expected Behavior:

- Clicking **Increase** → number goes up by 1
- Clicking **Decrease** → number goes down by 1

Part 2: Show / Hide Text

Task: Create a button that toggles visibility of a paragraph.

Requirements:

- Button text should change dynamically:
    - "Show Text" when hidden
    - "Hide Text" when visible

- Paragraph should appear/disappear

AI Prompt Help: "How do I toggle a paragraph visibility using useState in React?"

Part 3: Dynamic Message Switcher

Task: Display different messages when clicking a button.

Requirements:

- Start with message: "Hello!"
- Button: **Change Message**
- Cycle through messages like:
    - "Hello!"
    - "Welcome to React!"
    - "Keep Learning!"

AI Prompt Help: "How can I cycle through multiple messages using useState in React?"

Bonus Challenge (Optional)

Enhance your app by adding:

- Reset button for counter
- Animation when text appears/disappears
- Disable decrement when value is 0
- Color change based on state value

Reflection Questions

1. What is useState used for in React?
2. Why does React re-render when state changes?
3. What happens if you modify a variable without using state?
4. How did AI help you understand or build the components?

5.9. React Forms using States

When using React state in a form, you typically create a piece of state for each input field and update it as the user types. You can initialize state with **useState**, then bind the state value to the input's value prop to make it a controlled component. To keep the state in sync, attach an **onChange** handler that updates the corresponding state with **event.target.value**. On form submission, you can access all the current state values to process or send them to an API. This pattern ensures your form inputs always reflect React's state, making the data predictable, easy to validate, and straightforward to manage.

5.9.1. Handling Forms

Handling forms is about how you handle the data when it changes value or gets submitted.

- In HTML, form data is usually handled by the DOM.
- In React, form data is usually handled by the components. When the data is handled by the components, all the data is stored in the component ***state***.
- You can control changes by adding event handlers in the `onChange` attribute.
- We can use the `useState` Hook to keep track of each input value and provide a "single source of truth" for the entire application.

5.9.2. The useState statement

The React `useState` Hook allows us to track state in a function component. state generally refers to data or properties that need to be tracked in an application.

To use the `useState` Hook, we first need to `import` it into our component.

```
import { useState } from "react";
```

Notice that we are destructuring `useState` from `react` as it is a named export. The term destructing is the process of extracting the require data from a component.

For example, the old way of assigning array items to a variable:

```
const vehicles = ['mustang', 'f-150', 'expedition'];
// old way
const car = vehicles[0];
const truck = vehicles[1];
const suv = vehicles[2];
```

Here is the new way of assigning array items to a variable with destructing:

```
const vehicles = ['mustang', 'f-150', 'expedition'];
// with destructing
const [car, truck, suv] = vehicles;
```

5.9.3. Initialize useState

We initialize our state by calling useState in our function component. useState accepts an initial state and returns two values:

- The current state.
- A function that updates the state.

Example : Initialize state at the top of the function component.

```
import { useState } from "react";

function myForm() {

   const [name, setName] = useState('')

}
```

Example 5) Create a form using React JS to collect a **username, password, gender, and comments**.

For this app, we can create a form with an input control to collect a username:

App.js

```
import './App.css';
import React, {useState} from 'react';

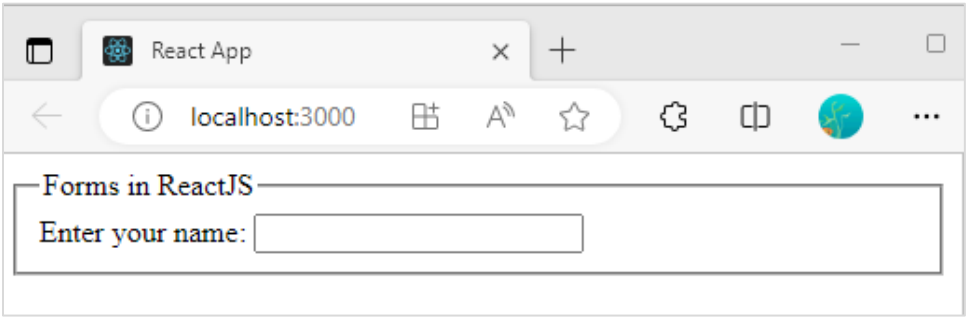
function App() {

return (
    <div className="App">
      <form className='myform' >
        <fieldset>
          <legend>Forms in React JS</legend>
          <div className='formgroup'>
              <label for="name">Enter username: </label>
```

```
                <input
                      type='text'
                      id='name'
                      className='inputstyle'
                      name='username'
                      placeholder='Type the username...'
                />
            </div>
    </div>
  );
}

export default App;
```

The internet browser should look as:



This will behave like a standard form submission—the form will submit and the page will refresh. However, this isn't typically what we want in React.

Instead, we aim to prevent the default behavior so React can handle and control the form submission.

Notice that again, we are de-structuring the returned values from `useState`.

- The first value, `name`, is our current state.
- The second value, `setName`, is the function that is used to update our state.

App.js

```
import './App.css';
import React, {useState} from 'react';

function App() {
  // set initial state for username
  const [username, setUsername] =  useState("")

return (
    <div className="App">
      <form className='myform' >
        <fieldset>
          <legend>Forms in React JS</legend>
          <div className='formgroup'>
              <label for="name">Enter username: </label>
              <input
                    type='text'
                    id='name'
                    className='inputstyle'
                    name='username'
                    placeholder='Type the username...'
                    onChange ={(e)=>setName(e.target.value}
              />
          </div>
    </div>
  );
}

export default App;
```

5.9.4. Submitting Forms

You can control the submit action by adding an event handler in the `onSubmit` attribute for the `<form>`:

```
import './App.css';
import React, {useState} from 'react';

function App() {
```

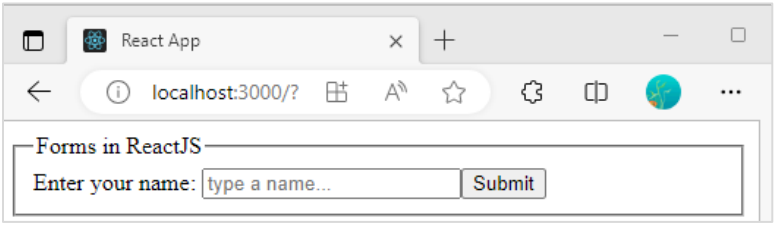
```
  // set initial state for username
  const [username, setUsername] =  useState("")

  // function submit form
  const submitform = function(event){
     event.preventDefault()
     alert(`Entered name ${username}`)
  }

  return (
    <div className="App">
      <form className='myform' >
        <fieldset>
          <legend>Forms in React JS</legend>
          <div className='formgroup'>
              <label for="name">Enter username: </label>
              <input
                    type='text'
                    id='name'
                    className='inputstyle'
                    name='username'
                    placeholder='Type the username...'
                    onChange ={(e)=>setName(e.target.value}
              />
          </div>
          {/** submit form */}
          <input type='submit' className='btnsubmit'/>
      </fieldset>
    </div>
  );
}

export default App;
```

The browser will look as follows:



5.9.5. Multiple Input Fields

You can control the values of more than one input field by adding a `name` attribute to each element.

- We will initialize our state with an empty object. To access the fields in the event handler use the `event.target.name` and `event.target.value` syntax.
- To update the state, use square brackets [bracket notation] around the property name.

Notice that again, we are destructuring the returned values from `useState`.

- The first value, `inputs`, is our current state.
- The second value, `setInputs`, is the function that is used to update our state.

These names are variables that can be named anything you would like.

Lastly, we set the initial state to an empty component: `useState({})`

Continuation of example 5) In the App.js file for example 5, we'll add another input field to capture a number, and then use JavaScript's spread method to handle multiple inputs. The modification the **App.js** file will be as follows:

```
import './App.css';
import React, {useState} from "react"

function App() {
  // set the initial state
  const[count, setCount] = useState(0)

  // set state for multiple inputs. destructing useState
  // useState({}) --> set a list of states
  const [inputs, setInputs] = useState({})

  // function to open an alert dialog when the form is submitted
  const submitform = function(event){
    event.preventDefault()
```

```
  alert(`Welcome to React State ${inputs.username}.
  Your lucky number is ${inputs.luckynumber}`)
}

// function to handle the changes in all inputs
const handle_changes =  function(event){
    const name = event.target.name
    const value = event.target.value
    // spread statement
    setInputs(values => ({...values, [name]:value}))
}
return (
  <div className="App">
    <form  onSubmit={submitform}>
      <fieldset>
        <legend>Forms in ReactJS</legend>
        <section>
          <label for="username">Enter your name: </label>
          <input
            type='text'
            id='username'
            name='username'
            value = {inputs.username}
            onChange={handle_changes}
          />
        </section>
        <section>
          <label for="luckynumber">Enter a lucky number:
          </label>
          <input
            type='number'
            id='luckynumber'
            name='luckynumber'
            value={inputs.luckynumber}
            onChange={handle_changes}
          />
```

```
          </section>

          {/**submit form */}
          <input type='submit' className='btnsubmit'/>
        </fieldset>
      </form>
      {/**test data in the form inputs */}
      <p>Name = {inputs.username}</p>
      <p>Lucky number= {inputs.luckynumber}</p>
    </>
  );
}
export default App;
```

Textarea

The textarea element in React is slightly different from ordinary HTML. In HTML the value of a textarea was the text between the start tag `<textarea>` and the end tag `</textarea>`.

Syntax:

```
<textarea>
  Content of the textarea.
</textarea>
```

In React the value of a textarea is placed in a value attribute. We'll use the `useState` Hook to manage the value of the textarea:

```
function MyForm() {
  {/* -- textarea -- */ }
  const [textComments, setTextarea] = useState("");
  const submitComment = function (event) {
   setTextarea(event.target.value)
}
```

```
return (
  <form>
  <textarea
    value={textComments}
    onChange={submitComment}
  />
)
<form>
```

Continuation of example 5) In the App.js file for example 5, we are going to add a text area to collect comments. The added code to the **App.js** file will be as:

```
// set the state for textarea
const[textcomment, setTextcomment] = useState("")
// function to collect the value of the textarea
const submitted_comments = function(event){
   setTextcomment(event.target.value)
}
```

In return:

```
<section>
    <label for="comments">Any suggestions? </label>
    <textarea
              id='comments'
              value={textcomment}
              onChange={submitted_comments}
     />
 </section>
```

Select

A drop-down list, or a select box, in React is also a bit different from HTML.

in HTML, the selected value in the drop down list was defined with the `selected` attribute:

```
<select value={myGender} onChange={selectedGender}>
       <option value="Female">Female</option>
       <option value="Male">Male</option>
       <option value="Other">Other</option>
</select>
```

In React, the selected value is defined with a `value` attribute on the `select` tag to pass the selected value in the state.

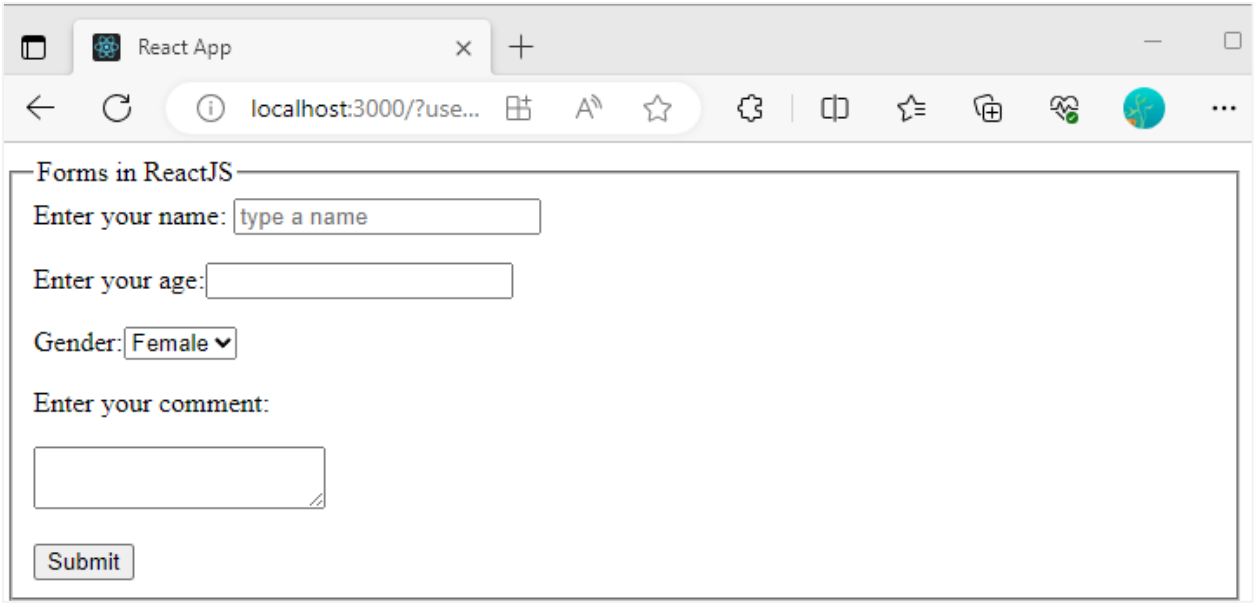
Continuation of example 5) In the App.js file for example 5, we can include a dropdown menu that allows the user to select a gender. The added code in **App.js** will look like this:

```
// set states for select gender
  const [myGender, setMygender] = useState("")
  // function to collect the selected gender
  const selectedGender = (event)=>{
    setMygender(event.target.value)
  }
```

In the return of the function component

```
{/** select a gender */}
        <div className='formgroup'>
          <label>Select a gender:
            <select value={myGender} onChange={selectedGender}>
              <option value="female">Female</option>
              <option value="male">Male</option>
              <option value="other">Other</option>
            </select>
         </label>
        </div>
```

In the browser:



The last design for this app will be to incorporate the information collected at the bottom of the form as:

```
{/**test the username collected after the submit button */}
<p className='printresult'>Username = {inputs.username}</p>
<p className='printresult'>Age = {inputs.age}</p>
<p className='printresult'>Gender = {myGender}</p>
<p className='printresult'>Comments = {textcomment}</p>
```

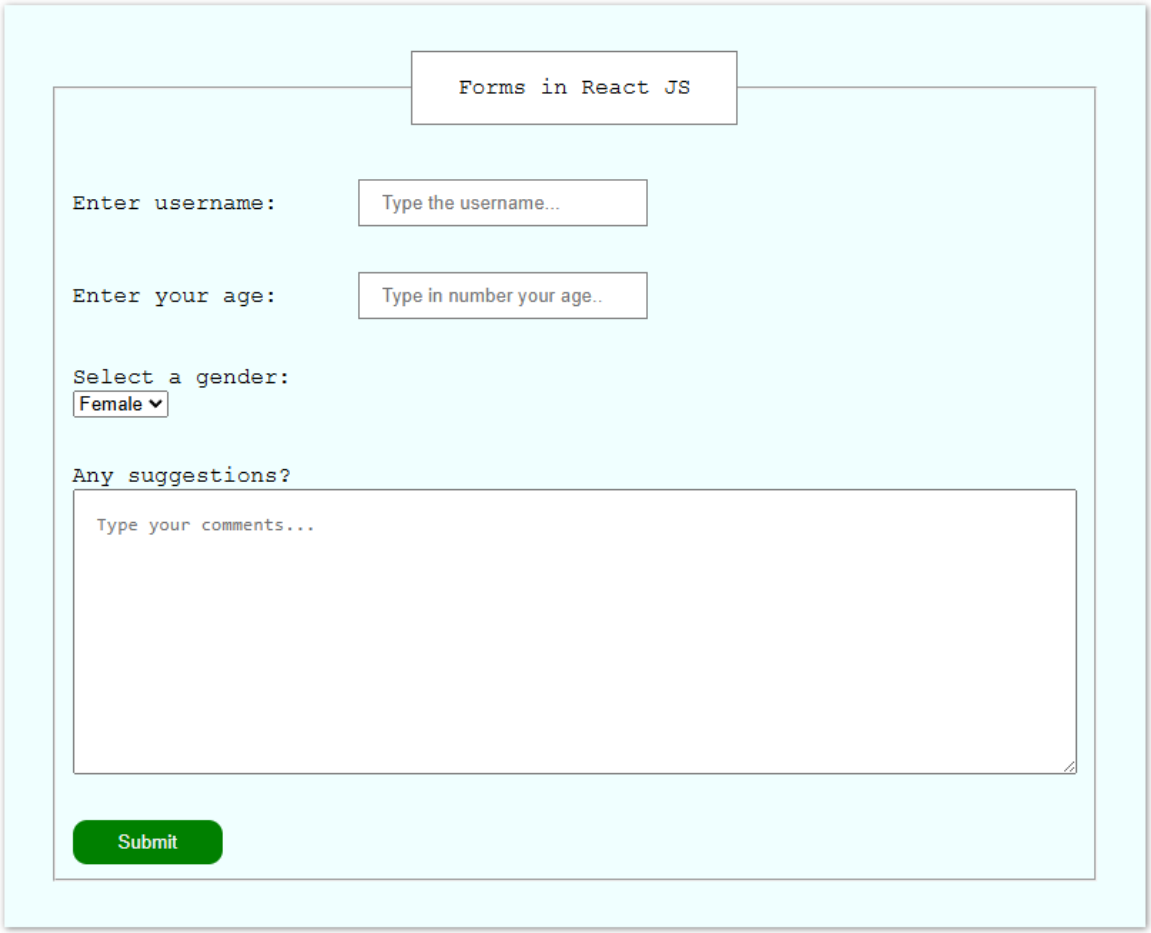
After applying the CSS styling, the app should look like this:

App.css

```
*{box-sizing: border-box;}
.App{
  font-family: 'Courier New', Courier, monospace;
}
.myform{
  width: 800px;
  padding: 2rem;
  box-shadow: 1px 1px 5px gray;
  margin: 3rem auto;
  background-color: azure;
}
```

```
.myform fieldset legend{
  padding: 1rem 2rem;
  text-align: center;
  background-color: white;
  border: solid gray 1px;
  margin-bottom: 2rem;
}
.inputstyle{
  padding: 0.5rem 1rem;
  border: solid gray 1px;
}
.formgroup{
  margin-bottom: 2rem;
}
label{
  width: 200px;
  display: inline-block;
}
textarea{
  display: block;
  width: 100%;
  height: 200px;
  padding: 1rem;
}
.btnsubmit{
  border: none;
  background-color: green;
  color:whitesmoke;
  padding: 0.5rem 2rem;
  border-radius: 10px;
}
.btnsubmit:hover{
  background-color: crimson;
  font-weight: bolder;
  box-shadow: 0px 0px 3px 1px gray;
  cursor: pointer;
}
```

The final styled version of the form should look like the following:




```
Username =

Age =

Gender =

Comments =
```

© 2026. This OER resource is licensed under a Creative Commons license (BY-NC-SA). It may be freely shared and redistributed for non-commercial purposes, provided proper attribution is given and any derivatives are shared under the same license.

Unauthorized copying, reproduction, or distribution of this material, in whole or in part, without permission is not allowed.